
GAOL v5

*NOT Just Another
Interval Arithmetic Library*

Edition 5.0
Last updated 22 September 2026

Jordan Ninin
Frédéric Goualard

ENSTA, France — LS2N, Nantes Université, France

Copyright © 2001 Swiss Federal Institute of Technology, Switzerland

Copyright © 2002-2016 LINA UMR CNRS 6241, France

Copyright © 2017-2026 LS2N UMR CNRS 6004, France

Copyright © 2026 ENSTA, France

All Trademarks, Copyrights and Trade Names are the property of their respective owners even if they are not specified below.

Part of the work was done while Frédéric Goualard was a postdoctorate at the *Swiss Federal Institute of Technology, Lausanne, Switzerland* supported by the *European Research Consortium for Informatics and Mathematics* fellowship programme.

This is edition 5.0 of the documentation of GAOL v5, which follows the manual of GAOL 4 by Frédéric Goualard. It is consistent with version 5.0.0 of the `gao1` library.

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

GAOL IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THIS SOFTWARE IS WITH YOU. SHOULD THIS SOFTWARE PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

Recipriversexcluson. n. *A number whose existence can only be defined as being anything other than itself.*

Douglas Adams, 1982
Life, the Universe and Everything

Contents

Copyright	ix
1 Introduction	1
1.1 GAOL v5	1
1.2 About this manual	2
2 Installation	3
2.1 Supported platforms	3
2.2 Getting the software	3
2.3 Prerequisites	4
2.3.1 Mandatory tools	4
2.3.2 Compilers and options refused	4
2.3.3 Tools for maintainers	4
2.3.4 Optional tools	5
2.4 Building and installing with CMake	5
2.4.1 Options	5
2.4.2 The archive of the sources and the packages	6
2.5 Building and installing with autotools	7
2.6 Building and installing with meson	8
2.7 Tests	8
2.8 What is installed	9
3 An overview of GAOL	11
3.1 Compiling a program using GAOL	12
3.1.1 From CMake	13
3.1.2 From pkg-config	13
3.2 The rounding direction	13
3.3 The namespaces	14
3.4 Common errors	15
3.4.1 Floating-point arithmetic and rounding	15
3.4.2 Zero	16
3.4.3 Both namespaces	16
3.4.4 The flags of interval arithmetic	16
4 Initialization and cleanup	17
5 Interval creation and assignment	19
5.1 Constructors	19
5.2 Straight assignment	21
5.3 Assignment combined with an operation	22
6 Interval constants	25

7	Interval relations	27
7.1	Set relations	27
7.2	Certainly relations	30
7.3	The order of IEEE 1788-2015	31
7.4	Relational Symbols	32
7.5	Interval-specific relations	33
8	Interval Arithmetic	37
8.1	Functional Arithmetic	38
8.1.1	Basic operations	38
8.1.2	Powers and roots	39
8.1.3	Exponentials and logarithms	41
8.1.4	Trigonometric functions	42
8.1.5	Hyperbolic functions	44
8.1.6	Other functions	45
8.2	Relational Arithmetic	45
8.2.1	$(n + 1)$ -ary relational functions	45
8.3	Cancellative addition and subtraction	48
9	Interval functions	49
9.1	Integer functions	52
9.2	Splitting methods	53
9.3	Union and intersection	54
10	Input/output	55
10.1	Reading intervals	55
10.1.1	Input format	55
10.2	Writing intervals	58
10.2.1	Converting intervals to strings	58
10.2.2	Output format	59
10.2.3	Choosing the number of digits to display	60
10.2.4	Example	61
11	The names of IEEE 1788-2015	63
11.1	Where the standard and GAOL differ	64
11.2	The operations	64
11.3	Reading with the names of the standard	66
11.4	What is not provided	66
12	Accuracy of the operations	67
12.1	Accuracy modes	67
12.2	Tightness of each operation	68
13	Floating-point numbers	71
13.1	Floating-point constants	71
13.2	Floating-point functions	71
14	Manipulating the FPU	73
14.1	Rounding functions	73
14.2	Saving the rounding direction	73
15	Version information	75
16	Additional functions	77

17 Error handling	79
17.1 Exceptions	80
17.1.1 The <code>gaol_exception</code> exception	80
17.1.2 The <code>input_format_error</code> exception	81
17.1.3 The <code>unavailable_feature_error</code> exception	81
17.1.4 The <code>invalid_action_error</code> exception	81
17.2 Warnings	82
18 Debugging facilities	83
19 Profiling	85
19.1 The <code>timepiece</code> class	86
19.1.1 Methods of the <code>timepiece</code> class	86
20 Additional Documentation	89
20.1 Documentation on GAOL	89
20.2 References	89
21 Reporting bugs	91
22 Contributors	93
23 Moving a program from GAOL 4 to GAOL v5	95
23.1 Compiling and linking	95
23.2 Results that change	96
Library Copying	99

Copyright

GAOL is distributed under the GNU Library General Public License, version 2 (see Section 23.2, page 99). The copyright for the initial version—named `cell`—(year 2001) is owned by the *Swiss Federal Institute of Technology*, Lausanne, Switzerland. The copyright for the following versions (from 2002 onward) is owned by the *Laboratoire d’Informatique de Nantes-Atlantique*, then by the *Laboratoire des Sciences du Numérique de Nantes*, France.

The files GAOL v5 adds carry the copyright of ENSTA, France, and are distributed under the same license as GAOL; the files of GAOL that GAOL v5 changes keep the copyright of their authors.

GAOL v5 computes its elementary functions with CORE-MATH [13], whose sources are distributed with GAOL (in `3rd/math-core`) and compiled into its library. CORE-MATH is distributed under the MIT license, which is reproduced with the license of GAOL (see Section 23.2, page 99).

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Gloudu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

1

Introduction

GAOL[†] is a C++ library to perform arithmetic with floating-point intervals. The development of GAOL was initiated at the *Swiss Federal Institute of Technology*, Lausanne, Switzerland, while F. Goualard was a post-doctorate fellow supported by the *Swiss National Science Foundation*. It started as a limited version of *Jail* (now *halloween*), a templated C++ interval library developed during Goualard’s PhD.

To our knowledge, a unique feature of GAOL among the C++ interval arithmetic libraries available when it was written is the implementation of *relational arithmetic operators* required by interval constraint arithmetic software (see Section 8.2, page 45). Hence, the game of the name: GAOL is *not* JAIL (*Just Another Interval Library*). The standard for interval arithmetic, IEEE 1788-2015 [9], has since required such operators, as its *reverse functions*.

[†] For those readers who are not native English speakers, “gaol” should be pronounced *jäl*, like the word “jail”, of which it is a, chiefly British, variant.

[9] IEEE. *IEEE Standard for Interval Arithmetic*. Institute of Electrical and Electronics Engineers, 2015. IEEE Std 1788-2015, doi:10.1109/IEEESTD.2015.7140721.

1.1 GAOL v5

GAOL v5, written by Jordan Ninin at ENSTA, continues GAOL as Frédéric Goualard developed it, from its version 4.2.2. It was written for Codac, whose intervals are built upon GAOL, and brings the changes GAOL needs to compile and compute right on every system it can: Linux, macOS and Windows, on x86, x86-64, ARM, 64-bit ARM and the other processors of Debian. What it brings, and this manual describes:

- **The tightest bounds for the elementary functions.** Every elementary function is bounded with CORE-MATH [13], which is correctly rounded in the rounding direction in effect: the value GAOL computes at a bound, rounding upward, *is* the upper bound, and the double below it the lower one. The bounds are the same on every architecture and with every compiler, and there is no mathematical library to install, to find or to link along with GAOL: CORE-MATH is compiled into it.
- **IEEE 1788-2015.** GAOL v5 follows the standard for interval arithmetic in the operations GAOL provides: the empty set, the infinite bounds, the domains of the functions, the comparisons and the interval literals. The few cases where it differs, mostly the `pow` of the namespace `gaol`, are stated where the operations are described, and the tightness of each operation is documented, as the standard requires (Chapter 12).

[13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Gloudu. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

- **New operations**, which the standard requires or recommends: `exp2`, `exp10`, `log2`, `log10`, `fma`, the integer functions `sign`, `trunc`, `round_ties_to_even` and `round_ties_to_away`, fifteen recommended functions (`expm1`, `log1p`, `hypot`, `rsqrt`, `sinpi`...), the cancellative subtraction and addition, the radius, and the comparisons `less`, `strictly_less`, `is_entire` and `is_common_interval`; `atan2`, which GAOL declared, is implemented.
- **The names of the standard**. The namespace `gaol_ieee1788` gives each operation of the standard that GAOL provides the name and the argument order the standard gives it (Chapter 11).
- **The rounding direction is the business of GAOL**. Each operation sets the rounding direction upward when it is not, whichever direction the calling code left: the user no longer has to keep it upward (see Section 3.2, page 13).
- **Exact input and output**. Numbers are read exactly, and intervals written in decimal are rounded outward, whatever the C library; written in hexadecimal, an interval is read back bit for bit.
- **Three builds that agree**. A CMake build, which is the one to use, and the autotools and meson builds of GAOL, which configure GAOL the same way; tests of the bounds GAOL computes against their exact values; a continuous integration on Linux, macOS and Windows, with the compilers and on the processors listed in Section 2.1.

A program written for GAOL 4 needs a few changes, which Chapter 23 lists: first of all, `gaol/gaol.h` no longer opens the namespace `gaol`.

1.2 About this manual

This document is both a manual and a reference to use GAOL. It assumes a prior knowledge of interval arithmetic. Refer to the books and papers by Goldberg, Neumaier, and others [2, 8, 10, 1, 11] for a basic presentation of floating-point arithmetic, interval arithmetic and the use thereof.

The main entry point for interval arithmetic on the Web is the *Interval Computation site* of Vladik Kreinovich, <http://www.cs.utep.edu/interval-comp/>.

The pages of the directory `doc` of the sources give the details of each change GAOL v5 brings, with its measures (see Section 20, page 89). The mark [GAOL v5] follows what behaves otherwise than in GAOL 4.

Classes, methods, functions, macros, constants and variables available in the library but not described in this document are likely to change or to be removed. Consequently, they should be used with caution, if at all. Among them are the intervals of floats, `gaol::intervalf` and `gaol::interval2f`, which are unfinished and which none of the builds compiles: only a developer of GAOL does, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.

[2] David Marc Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991.

[8] IEEE. IEEE standard for binary floating-point arithmetic. Technical Report IEEE Std 754-1985, Institute of Electrical and Electronics Engineers, 1985. Reaffirmed 1990.

[10] Ramon Edgar Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, N. J., 1966.

[1] Götz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press Inc., New York, USA, 1983. Traduit par Jon Rokne de l'original Allemand 'Einführung In Die Intervallrechnung'.

[11] Arnold Neumaier. *Interval methods for systems of equations*, volume 37 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 1990.

2

Installation

GAOL v5 is built with CMake, which is the build to use. The autotools and meson builds of GAOL are kept for those who use them: on a given machine with a given compiler, GAOL behaves the same whichever build configured it, the three giving it the same macros and the same flags.

2.1 Supported platforms

The continuous integration of GAOL v5 builds it and runs its tests on:

- Linux: Ubuntu 22.04, 24.04 and 26.04 on x86-64 and 64-bit ARM, with GCC and Clang; Debian 12 and 13 on x86-64, 64-bit ARM and 32-bit ARM (armhf), and Debian 12 on 32-bit x86; manylinux_2_28 on x86-64 and 64-bit ARM; Alpine (musl) on x86-64 and 64-bit ARM; Debian 13 on s390x, ppc64le and riscv64;
- macOS 14, 15 and 26, on arm64 and x86-64, with AppleClang, the Clang of LLVM and GCC;
- Windows: Visual Studio 2022 and 2026 on x86, x64 and arm64; MinGW-w64 12 to 15; MSYS2 (UCRT64 with GCC, CLANG64 with Clang).

The bounds GAOL computes are the same on all of them.

2.2 Getting the software

GAOL v5 is available at <https://github.com/Jordan08/GAOL>. The GAOL of Frédéric Goualard, which GAOL v5 continues, is available at <https://github.com/goualard-f/GAOL>, and presented in the GAOL section of <https://frederic.goualard.net>.

GAOL comes as source code, to be compiled for the computer it is used on. The sources of CORE-MATH are part of it (3rd/math-core): there is nothing else to download.

2.3 Prerequisites

2.3.1 Mandatory tools

- A C++11 and C99 compiler: GCC, Clang, AppleClang, Visual C++ (Visual Studio 2022 or later), or GCC and Clang for Windows (MinGW-w64, MSYS2).
- CMake 3.14 or later; or, for the autotools build, a shell and `make`; or, for the meson build, meson and ninja.

No mathematical library is needed: CORE-MATH is compiled into GAOL.

2.3.2 Compilers and options refused

Each of the following gave bounds not enclosing the exact results. The three builds refuse the compilers when configuring, with a message naming what to use instead, and keep the options away by giving GAOL the flags of interval arithmetic (see Section 3.1, page 12); `gaol/gaol_config.h` refuses them again when compiling, for the code including the headers of GAOL too, all but the second one, which no macro of the compiler shows:

- Clang for 32-bit ARM processors, which does not honour the rounding direction there (GCC does);
- a compiler saying of `-frounding-math` “overriding currently unsupported rounding mode on this target”, as Clang 14 for 64-bit ARM (Clang 18 honours the rounding direction there);
- a MinGW-w64 whose `<fenv.h>` answers `fegetround()` from a state of its own, before mingw-w64 12 on x64 and before 11 on 32-bit x86: GAOL sets the rounding direction by writing the registers of the processor, and CORE-MATH reads `fegetround()` to know it;
- `-ffast-math`, `-Ofast` and `/fp:fast`, with which the compiler rounds to nearest and drops the checks of NaN and infinities;
- Visual C++ without `/fp:strict`, which then assumes rounding to nearest;
- doubles computed on the x87 unit of 32-bit x86 processors, in extended precision (without `-msse2 -mfpmath=sse`, or `/arch:SSE2`): CORE-MATH assumes every operation on doubles rounded to a double.

2.3.3 Tools for maintainers

GAOL uses code produced by GNU Flex and GNU Bison for parsing the expressions used to initialize an interval. The three builds compile the lexer and the parser committed with the sources, `gaol_interval_lexer.cpp`, `gaol_interval_parser.cpp` and `gaol_interval_parser.h`, generated by flex 2.6.4 and bison 3.5.1 from `gaol_interval_lexer.lpp` and `gaol_interval_parser.ypp`. No build generates them again by itself, and none needs flex nor bison. After changing one of the two files, the target `parser` of the three builds (`make parser`, `ninja parser`) generates the lexer and the parser again in `gaol`, with the flex and the bison of the system, and they are committed; `gaol/regenerate_parser.sh` runs the same commands without a build:

```
% sh gaol/regenerate_parser.sh
```

2.3.4 Optional tools

- `flex` and `bison`, to generate the lexer and the parser again (`make parser`);
- `bash`, and Python 3 with `numpy`, for `make perf` (see Section 2.7, page 8);
- `pdflatex`, `bibtex` and `makeindex`, to build this manual.

2.4 Building and installing with CMake

In the root directory of the sources:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=prefix \
        -DWITH_TESTS=ON
% cmake --build build --config Release
% cmake --build build --target test
% cmake --install build --config Release
```

The build compiles the thirty-six sources of CORE-MATH GAOL uses into `libgaol`, which is a static library unless `BUILD_SHARED_LIBS` is `ON`: there is nothing else to build and nothing else to install. The build type is `Release` unless another is given; brought in by a project that gives none (`add_subdirectory`, `FetchContent`), GAOL and CORE-MATH are compiled with `-O3` all the same (`/O2` with Visual C++).

2.4.1 Options

The options are given to the first command, as `-Doption=value`:

- `CMAKE_BUILD_TYPE`. `Release` by default; `Debug` builds GAOL without optimization, with debugging information, and enables the debugging facilities (see Section 18, page 83);
- `CMAKE_INSTALL_PREFIX`. Where `cmake -install` puts GAOL;
- `BUILD_SHARED_LIBS`. `OFF` by default; `ON` builds the shared library `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, as `configure` and `meson` do (always a static library with Visual C++);
- `CMAKE_POSITION_INDEPENDENT_CODE`. `ON` by default: GAOL is compiled as position-independent code (`-fPIC`), so that it can be linked into a shared library, as Python bindings; a project building GAOL with `FetchContent` that sets it is followed;
- `WITH_TESTS`. `OFF` by default: builds the unit tests of `tests`, which `make test` and `make check` run (see Section 2.7, page 8);
- `WITH_EXAMPLES`. `OFF` by default: builds the examples of `examples`, which `make check` runs;
- `GAOL_SIMD`. `ON` by default: the intervals are computed with SSE2 instructions on x86 processors (`-msse2 -msse3`), their two bounds in one register; not with Visual C++ nor on 32-bit Windows;

- **GAOL_FMA.** ON by default: GAOL and CORE-MATH are compiled with the fused multiply-add instructions of the processor, where the compiler has a flag for them and the machine building runs a program compiled with it: `-mfma` on x86 (not with GCC for Windows), `/arch:AVX2` with Visual C++ for x64, `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM; 64-bit ARM, POWER, s390x and RISC-V have them without a flag. The library then needs a processor with them (on x86, Intel Haswell and AMD Piledriver, 2012–2013, and later), and the code using GAOL is given the flag too; **OFF** builds GAOL for any processor of the architecture. The elementary functions are much faster with them: `sin` and `cos` 2.4 times, `log` 1.6 times (Intel i7-1185G7, Clang 18.1);
- **GAOL_ASM.** ON by default: GAOL’s assembly code, where it has some (`GAOL_USING_ASM`);
- **GAOL_VERBOSE_MODE.** OFF by default: writes a line on the standard error when GAOL initializes and cleans up;
- **GAOL_PRESERVE_ROUNDING.** OFF by default: each operation restores the rounding direction it found, rather than leaving it upward (see Section 3.2, page 13);
- **GAOL_COVERAGE.** OFF by default: compiles GAOL and its tests with the counters of gcov, and adds the target `coverage`, which runs the tests and writes the report of their coverage (GCC and Clang, with `Debug`);
- **GAOL_U128_EMULATION.** OFF by default: computes the 128-bit integer of the accurate phases of CORE-MATH with two 64-bit halves, as where the compiler has no 128-bit type (Visual C++, and GCC for 32-bit targets).

The CMake build always reports errors by exceptions (see Section 17.1, page 80).

Interprocedural optimization is not offered, and `gaol::gaol` carries no such flag: with GCC, it moves floating-point operations across the changes of the rounding direction, and GAOL then gives bounds that do not enclose the exact results.

2.4.2 The archive of the sources and the packages

CPack makes them from the CMake build, in the directory of the build:

```
% cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/usr -DGAOL_FMA=OFF
% cmake --build build --config Release
% cmake --build build --target package_source
% cmake --build build --target package
```

- `package_source` gives `gaol-version.tar.gz`, the sources, which the three builds build. CPack takes the source tree as it is, less what git ignores and the builds made within it: the archive is made from a clean checkout;
- `package` gives `gaol-version-system.tar.gz`, what `cmake -install` installs, to extract anywhere, and on Linux `libgaol-dev_version_arch.deb`, the same files in `/usr`.

The packages hold `libgaol` as the build compiled it: with **GAOL_FMA** ON, the default, only the processors that have the fused multiply-add instructions run it, and **-DGAOL_FMA=OFF** makes packages for any processor of the architecture.

2.5 Building and installing with autotools

In the root directory of the sources, or in another directory for a build outside the sources:

```
% ./configure --prefix=prefix --with-tests
% make
% make test
% make install
```

`configure` compiles the sources of CORE-MATH into `libgaol`, with the flags it gives to the C code of GAOL. It is committed, generated by `autoconf 2.69` from `configure.ac`: `make` does not regenerate it as long as the files keep the dates of the checkout. There is no `make dist`: the archive of the sources is made with CMake (see Section 2.4.2, page 6). `configure` accepts the following options:

- `--help`. Displays a list of all options. Note that only those described hereunder are supported;
- `--prefix=prefix-dir`. The root directory where the library will be installed. It defaults to `/usr/local`;
- `--libdir=lib-dir`. The directory where to put the libraries. It defaults to `prefix-dir/lib`;
- `--includedir=include-dir`. The directory where to put the header files. It defaults to `prefix-dir/include`;
- `--enable-optimize[=yes/no]`. Compiles GAOL and CORE-MATH with `-O3` and `NDEBUG`, and with the flags `-funroll-loops`, `-fomit-frame-pointer` and `-fexpensive-optimizations` where the compiler takes them, as the Release build of CMake; with `-O` otherwise. This option defaults to `yes`;
- `--enable-debug[=yes/no]`. Compiles GAOL as the Debug build of CMake: with `-g`, without optimization nor `NDEBUG` (`--enable-optimize` is then ignored), and with the debugging facilities (see Section 18, page 83). This option defaults to `no`;
- `--enable-simd[=yes/no]`. The SSE2 intervals on x86 processors, as `GAOL_SIMD` (see Section 2.4.1, page 5). This option defaults to `yes`;
- `--enable-fma[=yes/no]`. The fused multiply-add instructions of the processor, as `GAOL_FMA`. This option defaults to `yes`;
- `--enable-asm[=yes/no]`. GAOL's assembly code, as `GAOL_ASM`. This option defaults to `yes`;
- `--enable-verbose-mode[=yes/no]`. Writes a line on the standard error when GAOL initializes and cleans up. This option defaults to `no`;
- `--enable-preserve-rounding[=yes/no]`. Each operation restores the rounding direction it found (see Section 3.2, page 13). This option defaults to `no`;
- `--enable-exceptions[=yes/no]`. If enabled, errors are reported by throwing an exception (see Section 17.1, page 80); otherwise, by calling `gaol_error()`, which prints a message to the standard error channel, and aborting. This option defaults to `yes`;

- `--with-tests[=yes/no]`. Builds the unit tests of `tests`, which `make test` and `make check` run, as `WITH_TESTS`. This option defaults to `no`;
- `--with-examples[=yes/no]`. Builds the examples of `examples`, which `make check` runs, as `WITH_EXAMPLES`. This option defaults to `no`.

Note that, as usual, `--disable-xxx` is equivalent to `--enable-xxx=no`. Moreover, `--enable-xxx` is equivalent to `--enable-xxx=yes`.

The targets of the Makefile in the root directory are `all`, which builds `libgaol.a` and the shared library `libgaol.so.5.0.0` (soname `libgaol.so.5`), `test`, `check`, `perf` and `parser`, those of the three builds (see Section 2.7, page 8), `install`, `clean` and `distclean`. `make -C manual pdf` builds this manual and the manual of GAOL 4. `configure -help` ends with the address for the bug reports, `jordan.ninin@ensta.fr`, and the page of GAOL v5, <https://github.com/Jordan08/GAOL>.

2.6 Building and installing with meson

```
% meson setup build --prefix=prefix -Dwith-tests=true
% meson compile -C build
% meson test -C build
% meson install -C build
```

meson compiles the sources of CORE-MATH into `libgaol`, as CMake and `configure` do, and the lexer and the parser committed with the sources: neither flex nor bison is needed. `meson compile` needs meson 0.54; with an older one, `ninja -C build` builds GAOL as well. It builds `libgaol.a` and `libgaol.so.5.0.0`, as `configure`; with Visual C++, the static library alone, as CMake. The options are given to `meson setup`, as `-Doption=value`; the build type gives the optimizations and the debugging facilities, as with CMake (`-buildtype=debug` for the Debug build), and the others are those of `configure`:

Option	Default
<code>buildtype</code>	<code>release</code> : <code>-O3</code> , <code>NDEBUG</code> and the optimizations of <code>--enable-optimize</code>
<code>enable-simd</code>	<code>true</code>
<code>enable-fma</code>	<code>true</code>
<code>enable-asm</code>	<code>true</code>
<code>enable-verbose-mode</code>	<code>false</code>
<code>enable-preserve-rounding</code>	<code>false</code>
<code>enable-exception</code>	<code>true</code>
<code>with-tests</code>	<code>false</code> ; builds the unit tests
<code>with-examples</code>	<code>false</code> ; builds the examples
<code>with-doc</code>	<code>false</code> ; the target <code>pdf</code> then builds this manual and the manual of GAOL 4

2.7 Tests

Built with `WITH_TESTS` (CMake), `--with-tests` (`configure`) or `with-tests` (meson), the programs of `tests` compare the bounds GAOL computes with the exact results of the operations, independently of GAOL and of the floating-point environment: the exact results are computed with integers, or were computed with 2000 bits of precision. They check, among others, that the

basic operations are the tightest enclosures, that the elementary functions are the tightest or within one double of them, that every operation gives the same result whatever the rounding direction the calling code left, that numbers are read exactly and intervals written rounded outward, that the reverse functions are accurate in the sense of IEEE 1788-2015, and that each name of `gaol_ieee1788` is the operation of the standard it names. The unit tests of GAOL 4, which checked the results of the operations on chosen intervals with CppUnit, are among them, and need CppUnit no more. The page `doc/tests.md` says what each program checks.

The three builds have the same targets:

- `make test` (`meson test`, `ninja test` with meson) runs the unit tests;
- `make check` (`ninja check`) runs the unit tests and the examples of `examples`, built with `WITH_EXAMPLES`, `--with-examples` or `with-examples`, which check what they print;
- `make perf` (`ninja perf`) measures GAOL v5 on the benchmark of `doc/compare/performance.md` and writes its times into the tables, the other libraries keeping the times of the last whole run (Linux, with bash, and Python 3 with numpy);
- `make parser` (`ninja parser`) generates the lexer and the parser of the strings again, with flex and bison.

`tests/find_package` builds the same tests with an installed GAOL, and `tests/fetch_content` with a GAOL brought into a project by `FetchContent`. `gaol_performance` (`tests/performance.cpp`) measures the time per operation of GAOL's arithmetic and elementary functions.

2.8 What is installed

Each build installs:

- the headers, in `prefix/include/gaol`;
- the library, `libgaol`, which holds CORE-MATH: static with CMake, or shared with `BUILD_SHARED_LIBS`; static and shared with autotools and meson (static only with Visual C++). The shared library is `libgaol.so.5.0.0`, whose soname is `libgaol.so.5`, with the three builds;
- `gaol.pc`, for pkg-config, in the `pkgconfig` directory of the library directory (except the CMake build with Visual C++);
- with CMake, the CMake package of GAOL, which defines the target `gaol::gaol`.

Both `gaol.pc` and `gaol::gaol` carry the flags that the code including the headers of GAOL has to be compiled with (see Section 3.1, page 12).

3

An overview of GAOL

In this chapter, we will assume that GAOL has already been properly installed, and that the libraries and header files are accessible to your compiler.

Let us consider the following program to compute the range of the function

$$f(x, y) = (1 + (x + y)^2(19 - 14x + 3x^2 - 14y + 6xy + 3y^2)) \\ (30 + (2x - 3y)^2(18 - 32x + 12x^2 + 48y - 36xy + 27y^2))$$

for $x \in [-2, 2]$ and $y \in [-2, 2]$.

Example

```
1  #include <iostream>
2  #include <gaol/gaol.h>
3  using namespace gaol;
4
5  int main(void)
6  {
7      interval
8          x(-2,2),
9          y(-2,2), z;
10
11     z=(1+sqr(x+y)*(19-14*x+3*sqr(x)-14*y+6*x*y+3*sqr(y)))*
12       (30+sqr(2*x-3*y)*(18-32*x+12*sqr(x)+
13         48*y-36*x*y+27*sqr(y)));
14
15     std::cout << "z = " << z << std::endl;
16     gaol::cleanup(); // Always, after the last use of GAOL
17     return 0;
18 }
```

First, note that we have to include the `gaol/gaol.h` header file in order to use all the facilities provided by GAOL. The type `interval` and the functions of GAOL are in the namespace `gaol_core`, which a program names through the namespace `gaol`: the `using` directive of Line 3 imports it. The `gaol/gaol.h` header of GAOL 4 imported the namespace itself; that of GAOL v5 no longer does, and a program adds the directive, or names `gaol::interval` and `gaol::sqr` (see Section 3.3, page 14). [\[GAOL v5\]](#)

Nothing has to be called before using GAOL: `gaol::init()` runs by itself when the program starts, before `main()`, with every compiler (see Section 4,



page 17), and each operation of GAOL sets the rounding direction it needs (see Section 3.2, page 13). [GAOL v5]

The program calls `gaol::cleanup()` right after its last use of GAOL, at the latest at the end of `main()`, whatever the compiler. Unless GAOL is built to preserve the rounding direction, its initialization sets the rounding direction upward for the whole program, not only for the operations of GAOL, and `gaol::cleanup()` sets back the one the program started with. GAOL also calls it itself, but only when the program ends: without the call, the code after the last use of GAOL would compute its doubles rounded upward. [GAOL v5]

The `sqr(x)` function stands for *square of x* and is equivalent to `pow(x,2)`.

Let `f.cpp` be the name of the file containing the program above. To compile it, with the flags `pkg-config` gives (see Section 3.1, page 12), we type the following command:

```
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) -o f f.cpp \
    $(pkg-config --libs gaol)
```

GAOL is the only library to link, with the C mathematical library: CORE-MATH, which computes its elementary functions, is compiled into it.

Executing `f`, we obtain:

	Output
<code>z = [-56254330, 94177270]</code>	

We then know that $f(x, y)$ ranges over $[-56254330, 94177270]$ when x and y range over $[-2, 2]$ independently.

3.1 Compiling a program using GAOL

The operations of GAOL on intervals are inline: the code that includes the headers of GAOL has to be compiled with the *flags of interval arithmetic*, not only GAOL itself. [GAOL v5] With GCC and Clang, each where the compiler takes it:

- `-frounding-math -fno-fast-math -ffp-contract=off`, so that the compiler rounds each operation in the direction in effect, as written, and neither evaluates it at compile time in the default rounding nor contracts a multiplication and an addition into a fused one;
- `-msse2 -mfpmath=sse` on 32-bit x86, so that doubles are computed in double precision rather than on the x87 unit;
- `-msse2 -msse3` on x86 processors with the SSE2 intervals (`GAOL_SIMD`);
- `-ffloat-store` where doubles are still computed on the x87 unit;
- `-mfma` on x86, and `-mfpu=neon-vfpv4 -mfloat-abi=hard` on 32-bit ARM, where GAOL is compiled with the fused multiply-add instructions of the processor (`GAOL_FMA`): the code using GAOL then runs only on a processor with them, as GAOL itself.

With Visual C++, `/fp:strict`, and `/arch:AVX2` for x64 with `GAOL_FMA`. Each build installs these flags with GAOL, in `gaol::gaol` and `gaol.pc`. `gaol/gaol_config.h` refuses code compiled by Visual C++ without `/fp:strict`. GCC and Clang do not tell the code whether `-frounding-math` and `-ffp-contract=off` were given: there, it only refuses what contradicts them, `-ffast-math` and doubles computed on the x87 unit (see Section 2.3.2, page 4).

3.1.1 From CMake

Example

```
1 find_package(gaol REQUIRED)
2 target_link_libraries(my_target PRIVATE gaol::gaol)
```

`gaol::gaol` carries the include directory, the flags above and, for Visual C++, the definition `__GAOL_PUBLIC__`, GAOL being a static library. There is no other library to link. A library whose headers include those of GAOL links `gaol::gaol PUBLIC`, so that its own users get the flags, and its CMake package finds GAOL again (`find_dependency(gaol)`).

A project can also build GAOL for itself, with the options it wants:

Example

```
1 include(FetchContent)
2 FetchContent_Declare(gaol GIT_REPOSITORY
3     https://github.com/Jordan08/GAOL.git GIT_TAG master)
4 FetchContent_MakeAvailable(gaol)
5 target_link_libraries(my_target PUBLIC gaol::gaol)
```

GAOL is then a target of the project, built with it, and `cmake -install` of the project installs it with it; nothing is downloaded beyond the sources of GAOL. GAOL is compiled as position-independent code, so that `gaol::gaol` can be linked into a shared library.

3.1.2 From pkg-config

Each build installs `gaol.pc` in the `pkgconfig` directory of its library directory, except the CMake build with Visual C++. Its `Cflags` carries the flags above with the include directory, and its `Libs` GAOL itself with the C mathematical library:

```
% export PKG_CONFIG_PATH=prefix/lib/pkgconfig
% c++ -std=c++11 -O2 $(pkg-config --cflags gaol) program.cpp \
    $(pkg-config --libs gaol)
```

Built by CMake with its default options on x86-64 Linux, GAOL gives, on one line:

```
% pkg-config --cflags --libs gaol
-frounding-math -fno-fast-math -ffp-contract=off
-msse2 -msse3 -mfma -Iprefix/include -Lprefix/lib -lgaol -lm
```

In a meson project, `dependency('gaol')` finds it.

3.2 The rounding direction

Floating-point interval arithmetic requires *outward rounding* in order to fulfill the *containment property*: for example, to add intervals $[a, b]$ and $[c, d]$, we compute $[\downarrow a + c \downarrow, \uparrow b + d \uparrow]$, where $\downarrow r \downarrow$ and $\uparrow r \uparrow$ return the greatest (resp. smallest) floating-point number smaller (resp. greater) than the real result of r . These two operations are performed by switching the rounding direction of the FPU towards, respectively, $-\infty$ and $+\infty$.

On most platforms, switching the rounding direction is costly. However, it is possible to cut down the number of switches by relying on the property that $\downarrow -r \downarrow = -\uparrow r \uparrow$. Consequently, one can replace nearly all downward rounding operations by upward rounding ones by negating appropriately twice the

Definition 1 (Round down/up)

Given $\mathbb{R}$ the set of real numbers and $\mathbb{F}$ the set of floating-point numbers (*double*), we have, for all $x \in \mathbb{R}$:

$$\begin{aligned}\downarrow x \downarrow &= \max\{y \in \mathbb{F} \mid y \leq x\} \\ \uparrow x \uparrow &= \min\{y \in \mathbb{F} \mid y \geq x\}\end{aligned}$$

operations performed. GAOL stores the lower bound of an interval negated, and computes both bounds rounding upward: the rounding direction can then be set towards $+\infty$ once and for all.

GAOL 4 put on the user the burden to ensure that the rounding direction be always set towards $+\infty$ before any computation involving intervals, which it called the *trust rounding mode*. GAOL v5 no longer does: [\[GAOL v5\]](#) *each operation sets the rounding direction upward when it is not*, and leaves it upward, whichever way GAOL is built. The bounds are then right whatever rounding direction the calling code left. The check is an addition, $1 + 2^{-60}$, which is above 1 only when rounded upward, rather than a reading of the rounding direction, which cost far more on some platforms: under Rosetta 2, reading it made an addition of intervals take 800 ns rather than 2.8 ns. Each function of intervals checks the direction once, and the functions it calls on doubles take it to be upward already.

Code that needs its own rounding direction after the operations of GAOL builds GAOL with the option `GAOL_PRESERVE_ROUNDING` of CMake, which is `--enable-preserve-rounding` with autotools and `-Denable-preserve-rounding=true` with meson: each operation then also restores the rounding direction it found, which makes the arithmetic operations several times slower. It is off by default in the three builds.

Without it, the rounding direction stays upward from the initialization of GAOL on, for the whole program, and `cleanup()` sets back the one that the initialization found: every program calls `gaol::cleanup()` right after its last use of GAOL (see Section 4, page 17). [\[GAOL v5\]](#)

Note. The rounding direction GAOL leaves applies to the rest of the program too. With the GNU C library, `printf()` and the streams of C++ round the decimal digits they write of a double in the direction in effect: after an operation of GAOL, `std::cout << pi_dn` writes 3.141592653589794 with 16 digits, and 3.141592653589793 once `round_nearest()` has been called. The intervals themselves are written by GAOL rounded outward, whatever the direction (see Section 10.2, page 58).

3.3 The namespaces

The type `interval` of GAOL and its functions are in the namespace `gaol_core`. A program names them through one of two namespaces, which `gaol/gaol.h` declares and does not open: [\[GAOL v5\]](#)

- `gaol`, GAOL under its own names, which this manual uses;
- `gaol_ieee1788`, GAOL under the names of IEEE 1788-2015 (Chapter 11).

The two namespaces hold `interval`, and the functions that have the same name and the same meaning in both (`sin`, `exp`, `sqrt`, `min`...) are the same functions, those of `gaol_core`. `pow` and `textToInterval` are the ones whose meaning differs: each namespace has its own, and none is in `gaol_core`, the namespace where argument-dependent lookup looks for a call `pow(x, y)` on an interval, so that each namespace finds its own only. A program opens one of the two namespaces, not both: with both open, `pow(x, y)` is ambiguous, and so is `textToInterval(s)`, which reads the names of GAOL in `gaol` and those of the standard in `gaol_ieee1788` (see Section 5.1, page 19).

The `pow` of `gaol` takes the integer power for an integer exponent, a negative base included, and the `pow` of IEEE 1788-2015 for any other exponent

(see Section 8.1.2, page 39). In `gaol_core`, its versions are named apart: `gaol_pown()`, `gaol_uipow()`, `gaol_pow_real()`, `gaol_pow_hybrid()`.

3.4 Common errors

In this section, we will review common errors made when using GAOL in the—*forlorn?*—hope that it will help prevent users from making them.

3.4.1 Floating-point arithmetic and rounding

Programming with floating-point numbers is one of the few activities where one must always consider ones compiler defiantly. For example, let us consider the following piece of code:

Example

```

1 #include <gaol/gaol> // We do not import the gaol namespace
2
3 using gaol::interval;
4
5 int main(void) {
6     interval one_tenth(0.1); BEWARE: wrong !
7
8     [some code using one_tenth]
9     gaol::cleanup();
10 }
```

Though a rational perfectly representable in decimal, `0.1` is not representable in binary (at least, not with a finite number of bits) and thus requires rounding. Obviously, the purpose of the user was to define an interval containing this value. However, `0.1` will be rounded *at compile time*, most certainly to the nearest representable floating-point number $\Downarrow 0.1 \Uparrow$. As a consequence, `one_tenth` will be a degenerate interval containing only $\Downarrow 0.1 \Uparrow$, and the *containment property* will be violated.

The right way to deal with rational constants that might not be perfectly representable as floating-point numbers is to stringify them, and to read them with `gaol::textToInterval()` (see Section 5.1, page 19), such that they can be correctly rounded downward and upward at *runtime*:

Example

```

1 #include <gaol/gaol>
2
3 using gaol::interval;
4
5 int main(void)
6 {
7     // OK: this is the right way
8     interval one_tenth = gaol::textToInterval("0.1");
9     // Another possible way
10    interval one_tenth2 = gaol::textToInterval("1/10");
11
12    [some code using one_tenth]
13    gaol::cleanup();
14    return 0;
15 }
```

Now, `one_tenth` will be the smallest floating-point interval enclosing `0.1`. An interval like this one, containing at most two consecutive floating-point

Definition 2 (Round to nearest)

Given $\mathbb{R}$ the set of real numbers and $\mathbb{F}$ the set of floating-point numbers (*double*), $\Downarrow x \Uparrow$ is, for $x \in \mathbb{R}$, the $y \in \mathbb{F}$ that minimizes $|x - y|$:

$$|x - \Downarrow x \Uparrow| = \min\{|x - y| \mid y \in \mathbb{F}\}$$

the one whose significand is even when two are equally near.

Definition 3 (canonical interval)

A non-empty interval $I = [a, b]$ is canonical if and only if $a \geq b^-$.

numbers, is called a *canonical interval*. Written in hexadecimal (see Section 10.2.2, page 59), which shows the bounds exactly, the two intervals are:

	Output
<code>interval(0.1)</code>	<code>= [0x1.999999999999ap-4, 0x1.999999999999ap-4]</code>
<code>one_tenth</code>	<code>= [0x1.999999999999p-4, 0x1.999999999999ap-4]</code>

3.4.2 Zero

`interval x(0, 0)` does not compile: the literal 0 is also a null pointer, which the constructor from two strings takes as well as the constructor from two doubles, and the call is ambiguous. Write `interval x(0.0, 0.0)`, `interval x(0)` or `interval::zero()`: `interval x(0)` compiles, the constructor from one string that made it ambiguous in GAOL 4 being gone. [GAOL v5]

3.4.3 Both namespaces

A program opens the namespace `gaol` or the namespace `gaol_ieee1788`, not both (see Section 3.3, page 14):

	Example
1	<code>#include <gaol/gaol.h></code>
2	<code>using namespace gaol;</code>
3	<code>using namespace gaol_ieee1788;</code>
4	
5	<code>gaol::interval x(1,2), y = pow(x,x); // Does not compile</code>

The compiler reports that the call of `pow` is ambiguous.

3.4.4 The flags of interval arithmetic

Compiled without the flags of interval arithmetic (see Section 3.1, page 12), the inline operations of GAOL may be evaluated by the compiler at compile time, rounded to nearest, or contracted into fused multiply-adds, and give bounds that do not enclose the exact results. `gaol/gaol_config.h` stops the compilation where it can tell:

	Output
	<code>% c++ -ffast-math -c program.cpp</code>
	<code>gaol/gaol_config.h: error: #error "GAOL cannot be</code>
	<code>compiled with -ffast-math (nor -Ofast): the bounds it</code>
	<code>computes would not enclose the exact results"</code>

Taking the flags from `gaol::gaol` or `gaol.pc` avoids the question.

4

Initialization and cleanup

GAOL initializes itself when the program starts, before `main()`: `init()` need not be called. Since Release 1.0 of GAOL, there is an automatic initialization and cleanup, so that GAOL works even when the program calls neither function. Since GAOL v5, the initialization is made with every compiler, Visual C++ included: each file including the headers of GAOL holds a static object whose constructor initializes GAOL the first time, as `<iostream>` initializes the standard streams, so that GAOL is initialized before the static objects that such a file defines after its `#include`. Each operation also sets the rounding direction upward itself (see Section 3.2, page 13), so that `init()` no longer has to be called again after the calling code changed it. [\[GAOL v5\]](#)

`cleanup()`, on the other hand, is to be called by the program right after its last use of GAOL, at the latest at the end of `main()`, with every compiler. [\[GAOL v5\]](#) Unless GAOL is built to preserve the rounding direction, the initialization leaves the rounding direction upward for the whole program, not only for the operations of GAOL, and `cleanup()` sets back the one the program started with. The automatic cleanup does so too, but only when the program ends, once `main()` has returned and the static objects are destroyed, or when the library is unloaded: until then, the code after the last use of GAOL (the rest of `main()`, the destructors of static objects, the functions registered with `std::atexit()`) computes its doubles rounded upward.



The direction set back is the one GAOL found when it initialized itself. It is not the one the program started with when an operation of GAOL came first: in the initialization of a static object of a file that uses GAOL without including its headers, through a library of the program for instance, or, with Clang, of an `inline` variable or a static member of a class template, whose initialization C++ does not order with the rest of the file.

The constants of GAOL (π , the masks of its SSE2 operations) are initialized when compiling: a static object of the program may compute intervals, before the files of GAOL are initialized, which with the static library of the CMake build come after those of the program, except with MinGW-w64. [\[GAOL v5\]](#)

```
bool init (int dbg_lvl = 0)
```

Initializes the variable `debug_level` (see Section 18, page 83) to the value of `dbg_lvl`.

Unless GAOL was built to preserve the rounding direction, the first call notes the rounding direction it finds, which `cleanup()` sets back, then sets the floating-point environment to its default one, then the rounding direction towards $+\infty$. In addition, it sets the number of digits to display for interval bounds to 16.

Returns `true` if the library was not already initialized and `false` otherwise. After its first call, the function sets the debugging level to its new value and, unless the rounding direction is preserved, the rounding direction upward again: the C runtime of Windows resets it when the program starts, after the library initialized itself.

bool `cleanup` (void)

Unless GAOL was built to preserve the rounding direction, sets back the rounding direction that the first call of `init()` found: the one of the program when GAOL initialized itself, to nearest when the program starts. On x86 processors, the x87 unit and the SSE instructions each get theirs back. The rest of the floating-point environment is left as it is, the exception flags raised since in particular. The rounding direction being one of each thread, it is set back in the thread calling `cleanup()`. [GAOL v5] The program calls it right after its last use of GAOL: an operation of GAOL after it sets the rounding direction upward again. The automatic cleanup calls it when the program ends, or when the library is unloaded, if the program did not, and frees what the initialization allocated, after the static objects of the program are destroyed: the expressions of `gaol/gaol_expression.h` still alive when the program calls `cleanup()` remain valid.

Returns `true` the first time it is called and `false` afterwards: only the first call sets back the rounding direction.

Example

```
1 #include <gaol/gaol.h>
2
3 int main(void)
4 {
5     gaol::init(1); // First level of debugging requested;
6                   // without it, GAOL initializes itself
7
8     [Some code using interval arithmetic]
9
10    gaol::cleanup(); // Always, right after the last use
11                    // of GAOL: sets back the rounding
12                    // direction
13    return 0;
14 }
```

5

Interval creation and assignment

The methods for creating an interval and assigning a new value to an already existing one are described in the following.

5.1 Constructors

One can create an interval with five constructors, or with `textToInterval()`:

- by providing its left and right bounds:

Example

```
1 interval x(l,r);
```

where l and r are `doubles` or of a type that is castable into a `double`. The result is the empty set when $[l, r]$ is not an interval: for a left bound $+\infty$, a right bound $-\infty$, bounds in the wrong order ($l > r$) and NaN bounds; [\[GAOL v5\]](#)

- by providing only one bound, for degenerate point intervals:

Example

```
1 interval x(v);
```

This is equivalent to: `interval x(v,v);`, and the empty set for an infinite or NaN v ;

- without providing any bound:

Example

```
1 interval x;
```

This is equivalent to:

Example

```
1 interval x(-GAOL_INFINITY,+GAOL_INFINITY);
```

where `GAOL_INFINITY` represents the infinity value of the `double` format (see Section [13.1](#), page [71](#));

- by copying an already existing interval (*copy constructor*):

Example

```
1 interval x(-12,12), y=x, z(x);
```

- by using a string representing an interval in the same format as the one used for input (see Section 10.1.1, page 55) for each bound: the left bound of the first one and the right bound of the second one are taken.

Example

```
1 // constructs x=[-4,4]
2 interval x("[-5,4]+1", "[4,6]-[2,3]");
```

`textToInterval(sl, sr)` of the namespace `gaol` is this constructor;

- by reading a string representing an interval in the same format as the one used for input (see Section 10.1.1, page 55), with `textToInterval()`:

interval `textToInterval` (const std::string& s)

Returns the interval the string *s* writes, read with the names of the functions of GAOL (see Section 10.1.1, page 55). It is in the namespace `gaol`, under the name `IEEE 1788-2015` gives it.

Example

```
1 interval x = textToInterval("[-23, inf]"),
2 y = textToInterval("[5*0.1+dmin, 89*sinh(2.1)]");
```

If the input string does not comply with the expected format, the exception `gaol::input_format_error` is thrown if the library was compiled with exception support, which the CMake build always is; in the absence of exception support, the `errno` variable is set to -1, and the `gaol_error()` function prints an error message before the program aborts (see Section 17, page 79). The `textToInterval()` of `gaol_ieee1788` reads the names of the functions of the standard, and returns the empty set instead (Chapter 11).

GAOL v5 has no constructor from one string: `textToInterval()` takes the place of the constructor `interval(const char*)` of GAOL 4, which converted any `const char*` implicitly: `interval(0)` was ambiguous, and `x + p` compiled for a null `p`, and crashed. [GAOL v5]

Caution. you have to be very careful when creating an interval from floating-point constants. Remember that a rational number that is perfectly representable in the decimal base may require rounding in the binary base. For example, if you write the following statement:

Example

```
1 interval x(0.1);
```

you will *not* have created an interval containing 0.1 since this number has an infinite expansion in the binary base (i.e. it is impossible to represent it perfectly whatever the size of the mantissa may be). As a consequence, the constant 0.1 has very likely been rounded to the nearest floating-point number at compile-time. In such a case, you have to read it from a string instead:

Example

```
1 interval x = textToInterval("0.1");
```

Note. The degenerate intervals $[+\infty, +\infty]$ and $[-\infty, -\infty]$ do not exist in GAOL v5. [GAOL v5] IEEE 1788-2015 has no such interval, and its constructor fails there and gives the empty set (10.5.8, 12.12.7); the constructors of GAOL give the empty set too, as those of IBEX do, for an infinite point and for all the cases above:

Example

```

1 cout << interval(GAOL_INFINITY) << ' '
2   << interval(-GAOL_INFINITY, -GAOL_INFINITY) << ' '
3   << interval(2,1) << ' '
4   << interval(1,GAOL_INFINITY) << endl;
5  [empty] [empty] [empty] [1, inf]
```

GAOL 4 built them as intervals of one infinite point, which its operators did not support. The intervals with one infinite bound, as $[1, +\infty]$, are intervals as any other.

Read from a string, the constant `inf`, an expression, stands for $[MAX, +\infty]$, `MAX` being the largest double, and `-inf` for $[-\infty, -MAX]$; the interval literals whose left bound is $+\infty$ or whose right bound is $-\infty$ are the empty set, as `numsToInterval` of IEEE 1788-2015 has it:

Example

```

1 cout << textToInterval("inf") << ' ' << textToInterval("[inf]") << ' '
2   << textToInterval("[-inf, -inf]") << endl;
3  [1.797693134862315e+308, inf] [empty] [empty]
```

The string `"<-inf, -inf">` throws `input_format_error`, its two bounds being different values (see Section 10.1.1, page 55).

5.2 Straight assignment

It is possible to assign a new value to an already existing interval in three different ways:

- by copying another interval:

Example

```

1 interval x(-12,12),
2           y; // Here, y is [-∞, +∞]
3 y = x; // Now, y is [-12, 12]
```

- by using a string whose format follows the one expected for input (see Section 10.1, page 55), which `textToInterval()` reads:

Example

```

1 interval x;
2 x = textToInterval("[-inf, 123]");
```

GAOL 4 assigned the string itself, `x = "[-inf, 123]"`, which GAOL v5 refuses, having no constructor from one string; [GAOL v5]

- by using a double:

Example

```

1 interval x;
2 x = 1234.5; // Now, x is [1234.5, 1234.5]
3 x = GAOL_INFINITY; // Now, x is empty
```

Both follow the rules of the previous section: assigning an infinity to an interval empties it. [\[GAOL v5\]](#)

Note that there is no method for modifying a bound of an interval since intervals must be considered as an atomic concept.

5.3 Assignment combined with an operation

The following assignment operators combine the value of the interval pointed to by **self** and the value of the right-hand side interval.

interval& interval::operator&= (const interval& I)

■ $\text{self} \leftarrow \text{self} \cap I$

Assigns to **self** the interval resulting from the intersection of **self** and *I*, which is the empty set when they are disjoint. [\[GAOL v5\]](#)

Example

```
1 interval x(-12,12);
2 x &= interval(-6,23); // Now, x is [-6, 12]
```

interval& interval::operator|= (const interval& I)

■ $\text{self} \leftarrow \square(\text{self} \cup I)$

Assigns to **self** the interval hull of the union of **self** and *I*.

Example

```
1 interval x(-12,12);
2 x |= interval(-6,23); // Now, x is [-12, 23]
```

interval& interval::operator+= (const interval& I)

interval& interval::operator+= (double d)

■ $\text{self} \leftarrow \text{self} + I$

■ $\text{self} \leftarrow \text{self} + d$

Assigns to **self** the interval resulting from adding **self** and *I* (resp. *d*).

Example

```
1 interval x(-12,12);
2 x += interval(-6,23); // Now, x is [-18, 35]
```

interval& interval::operator-= (const interval& I)

interval& interval::operator-= (double d)

■ $\text{self} \leftarrow \text{self} - I$

■ $\text{self} \leftarrow \text{self} - d$

Assigns to **self** the interval resulting from subtracting *I* (resp. *d*) from **self**.

Example

```
1 interval x(-12,12);
2 x -= interval(-6,23); // Now, x is [-35, 18]
```

interval& interval::operator*= (const interval& I)

interval& interval::operator*= (double d)

```

■ self ← self × I
■ self ← self × d

```

Assigns to **self** the interval resulting from multiplying **self** and *I* (resp. *d*).

Example

```

1 interval x(-12,12);
2 x *= interval(-6,23); // Now, x is [-276, 276]

```

```

interval& interval::operator/= (const interval& I)
interval& interval::operator/= (double d)

```

```

■ self ← self/I
■ self ← self/d

```

Assigns to **self** the interval resulting from dividing **self** by *I* (resp. *d*).

Example

```

1 interval x(-12,12);
2 x /= interval(-6,23); // Now, x is [-∞, +∞]
3 x /= interval::zero(); // Now, x is ∅

```

```

interval& interval::operator%=(const interval& I)
interval& interval::operator%=(double d)

```

Assigns to **self** the interval resulting from dividing **self** by *I* (resp. *d*), using a *relational division* (see Section 8.2, page 45).

Example

```

1 interval x(-12,12);
2 x %= interval(-6,23); // Now, x is [-∞, +∞]
3 x %= interval::zero(); // Now, x is [-∞, +∞]

```

The operators with a double compute as those with an interval, by the sign of the double; an infinite double gives the empty set, as **interval(d)** is empty. [GAOL v5]

6

Interval constants

For convenience, some useful intervals and some canonical intervals enclosing real constants are defined as static functions of the `interval` class:

Function	Value
<code>interval::emptyset()</code>	$\emptyset$
<code>interval::half_pi()</code>	$[\downarrow \frac{\pi}{2} \downarrow, \uparrow \frac{\pi}{2} \uparrow]$
<code>interval::minus_one_plus_one()</code>	$[-1, 1]$
<code>interval::negative()</code>	$[-\infty, 0]$
<code>interval::one()</code>	1
<code>interval::one_plus_infinity()</code>	$[1, +\infty]$
<code>interval::pi()</code>	$[\downarrow \pi \downarrow, \uparrow \pi \uparrow]$
<code>interval::positive()</code>	$[0, +\infty]$
<code>interval::two_pi()</code>	$[\downarrow 2\pi \downarrow, \uparrow 2\pi \uparrow]$
<code>interval::universe()</code>	$[-\infty, +\infty]$
<code>interval::zero()</code>	0

The enclosures of π , 2π and $\pi/2$ are the tightest ones. The empty set is held as an interval whose two bounds are NaN.

Example

```
1 cout << interval::emptyset() << ' ' << interval::pi() << endl;
2  [empty] [3.141592653589793, 3.141592653589794]
```


7

Interval relations

Interval relations may be divided into two groups. Given I and J two intervals, we have:

1. *set relations*: intervals I and J are considered as sets of reals. For example:

$$I = J \Leftrightarrow (\forall x \in I, \exists y \in J: x = y) \wedge (\forall y \in J, \exists x \in I: x = y)$$

Basically, two intervals are equal in that mode if they have the same bounds;

2. *certainly relations*: the relations must be true for any tuple of values in the intervals. For example:

$$I \leq J \Leftrightarrow (\forall x \in I, \forall y \in J: x \leq y)$$

Then, I is less than or equal to J in that mode if its right bound is at most the left bound of J .

GAOL 4 had a third group, the *possibly relations*, true if at least one tuple of values verifies the real relation, and a certain equality, `certainly_eq()` and `certainly_neq()`. They are gone from GAOL v5, and so are the operators `==` and `!=` on intervals: [GAOL v5] `certainly_neq()` was the negation of `certainly_eq()`, which is “possibly not equal”, so that $[3, 4]$ was certainly not equal to $[3, 4]$. IEEE 1788-2015 has neither. Its equality is `equal`, `set_eq()`; that no value of I equals a value of J is `disjoint`, `set_disjoint()`; and “possibly equal” is `!set_disjoint()`.

The relation symbols $<$, $\leq$, $>$ and $\geq$ are the certainly relations in every build (see Section 7.4, page 32).

The relations that IEEE 1788-2015 defines (Tables 10.3 and 10.4) follow its definitions, for the empty set and for the infinite bounds too: [GAOL v5] `equal`, `subset`, `interior`, `disjoint`, `precedes`, `strictPrecedes`, `less` and `strictLess`. Each method below says which one it is; the namespace `gaol_ieee1788` gives them their names (Chapter 11).

7.1 Set relations

```
bool interval::set_contains (const interval& I) const
bool interval::set_contains (double d) const
```

- $I \subseteq \text{self}$
- $d \in \text{self}$

Returns **true** if I (resp. $\{d\}$) is included in **self**: `subset( $I$ , self)`, and `isMember( $d$ , self)` of IEEE 1788-2015 for a finite d . d shall not be a NaN.

Example

```

1 interval x(-12,34), y(-12,5);
2
3 cout << x.set_contains(y) << ' '
4     << x.set_contains(interval::emptyset()) << ' '
5     << y.set_contains(x) << ' '
6     << interval::emptyset().set_contains(x) << ' '
7     << interval::emptyset().set_contains(interval::emptyset())
8     << endl;
9  true true false false true

```

`bool interval::set_strictly_contains (const interval&  $I$ ) const`
`bool interval::set_strictly_contains (double  $d$ ) const`

- I interior to **self**
- d in the interior of **self**

Returns **true** if I is interior to **self**, `interior( $I$ , self)` of IEEE 1788-2015: each bound of **self** is beyond that of I , an infinite bound being beyond the same infinite bound, and the empty set is interior to any interval, itself included. [GAOL v5] The second form returns **true** if d is strictly between the bounds of **self**.

Example

```

1 interval x(-10,12), y(-10, 11), z, t, u(10.5);
2
3 cout << boolalpha
4     << x.set_strictly_contains(y) << ' '
5     << z.set_strictly_contains(t) << ' '
6     << x.set_strictly_contains(u) << ' '
7     << interval::emptyset().set_strictly_contains(
8         interval::emptyset()) << ' '
9     << u.set_strictly_contains(interval::emptyset())
10    << endl;
11  false true true true true

```

`bool interval::set_disjoint (const interval&  $I$ ) const`

- $\text{self} \cap I = \emptyset$

Returns **true** if the intersection of **self** and I is empty: `disjoint` of IEEE 1788-2015.

Example

```

1 interval a(2,4), b(6,8);
2 cout << a.set_disjoint(b) << ' '
3     << interval::emptyset().set_disjoint(interval::emptyset());
4  true true

```

`bool interval::set_eq (const interval&  $I$ ) const`

■ $\forall x \in \text{self} \exists y \in I: x = y \wedge \forall y \in I, \exists x \in \text{self}: y = x$

Returns **true** if intervals **self** and **I** are equal when considered as sets of reals: equal of IEEE 1788-2015.

Example

```

1 cout << interval(4,8).set_eq(interval(4,8)) << ' '
2   << interval::emptyset().set_eq(interval::emptyset()) << endl;
3 true true

```

bool interval::set_neq (const interval& I) const

■ $\neg(\text{self} = I)$

Returns **true** if **self** and **I** are not equal when considered as sets of reals.

Example

```

1 cout << interval::universe().set_neq(interval::emptyset());
2 true

```

bool interval::set_le (const interval& I) const

■ **self** interior to **I**

Returns **true** if **self** is interior to **I**: **I.set_strictly_contains(self)**, interior(**self**, **I**) of IEEE 1788-2015. [\[GAOL v5\]](#)

Example

```

1 interval a(2,GAOL_INFINITY), b(1,GAOL_INFINITY);
2 cout << interval(-4.5,3).set_le(interval(-10,10)) << ' '
3   << interval::emptyset().set_le(interval(5,6)) << ' '
4   << interval::emptyset().set_le(interval::emptyset()) << ' '
5   << interval::universe().set_le(interval::universe()) << ' '
6   << a.set_le(b);
7 true true true true true

```

bool interval::set_leq (const interval& I) const

■ **self** $\subseteq I$

Returns **true** if the real set defined by **self** is included in **I**: **subset(self, I)** of IEEE 1788-2015.

Example

```

1 cout << interval(4.5,6).set_leq(interval(4.5,6)) << ' '
2   << interval(3.5,9).set_leq(interval(2,6)) << ' '
3   << interval::emptyset().set_leq(interval::emptyset());
4 true false true

```

bool interval::set_ge (const interval& I) const

■ **I** interior to **self**

Returns **true** if **I** is interior to **self**: **set_strictly_contains(I)**.

Example

```

1 cout << interval(-10,10).set_ge(interval(-4.5,3)) << ' '
2   << interval(1,2).set_ge(interval(1,2));
3 true false

```

bool interval::set_geq (const interval& I) const

■ **I** $\subseteq$ **self**

Returns **true** if the real set defined by **self** contains **I**: **set_contains(I)**.

7.2 Certainly relations

bool interval::certainly_le (const interval& I) const

■ $\forall x \in \text{self}, \forall y \in I: x < y$

Returns **true** if **self** is certainly strictly less than *I*: strictPrecedes of IEEE 1788-2015, true when either interval is empty. [\[GAOL v5\]](#)

Example

```
1 cout << interval(4,5).certainly_le(interval(6,9)) << ' '
2   << interval(4,5).certainly_le(interval(5,9)) << ' '
3   << interval::emptyset().certainly_le(interval(4,6)) << ' '
4   << interval(4,6).certainly_le(interval::emptyset());
5  true false true true
```

bool interval::certainly_leq (const interval& I) const

■ $\forall x \in \text{self}, \forall y \in I: x \leq y$

Returns **true** if **self** is certainly less or equal to *I*: precedes of IEEE 1788-2015, true when either interval is empty. [\[GAOL v5\]](#)

Example

```
1 cout << interval(4,5).certainly_leq(interval(6,9)) << ' '
2   << interval(4,5).certainly_leq(interval(5,9)) << ' '
3   << interval(5,9).certainly_leq(interval(4,5)) << ' '
4   << interval(4,8).certainly_leq(interval(5,9)) << ' '
5   << interval::emptyset().certainly_leq(interval(4,6)) << ' '
6   << interval(4,6).certainly_leq(interval::emptyset());
7  true true false false true true
```

bool interval::certainly_ge (const interval& I) const

■ $\forall x \in \text{self}, \forall y \in I: x > y$

Returns **true** if **self** is certainly strictly greater than *I*, or if either interval is empty.

Example

```
1 cout << interval(8,10).certainly_ge(interval(4,8)) << ' '
2   << interval(8,10).certainly_ge(interval(4,7)) << ' '
3   << interval::emptyset().certainly_ge(interval::emptyset());
4  false true true
```

bool interval::certainly_geq (const interval& I) const

■ $\forall x \in \text{self}, \forall y \in I: x \geq y$

Returns **true** if **self** is certainly greater or equal to *I*, or if either interval is empty.

Example

```
1 cout << interval(8,10).certainly_geq(interval(4,8)) << ' '
2   << interval::emptyset().certainly_geq(interval::emptyset());
3  true true
```

bool interval::certainly_positive (void) const

■ $\text{self} \subseteq [0, +\infty]$

Returns **true** if the left bound of **self** is greater or equal to zero, or if **self** is empty.

Example

```
1 cout << interval::emptyset().certainly_positive() << ' '
2   << interval(4,5).certainly_positive() << ' '
3   << interval(-0.0,6).certainly_positive() << ' '
4   << interval(-6,0).certainly_positive();
5  true true true false
```

bool interval::**certainly_strictly_positive** (void) const

■ $\text{self} \subseteq]0, +\infty]$

Returns **true** if the left bound of **self** is strictly greater than zero, or if **self** is empty.

Example

```
1 cout << interval::emptyset().certainly_strictly_positive() << ' '
2   << interval(4,5).certainly_strictly_positive() << ' '
3   << interval(-0.0,6).certainly_strictly_positive() << ' '
4   << interval(-6,0).certainly_strictly_positive();
5  true true false false
```

bool interval::**certainly_negative** (void) const

■ $\text{self} \subseteq [-\infty, 0]$

Returns **true** if the right bound of **self** is lower or equal to zero, or if **self** is empty.

Example

```
1 cout << interval::emptyset().certainly_negative() << ' '
2   << interval(4,5).certainly_negative() << ' '
3   << interval(-6,0).certainly_negative() << ' '
4   << interval(-6,-5).certainly_negative();
5  true false true true
```

bool interval::**certainly_strictly_negative** (void) const

■ $\text{self} \subseteq [-\infty, 0[$

Returns **true** if the right bound of **self** is strictly lower than zero, or if **self** is empty.

Example

```
1 cout << interval::emptyset().certainly_strictly_negative() << ' '
2   << interval(4,5).certainly_strictly_negative() << ' '
3   << interval(-6,0).certainly_strictly_negative() << ' '
4   << interval(-6,-5).certainly_strictly_negative();
5  true false false true
```

7.3 The order of IEEE 1788-2015

bool interval::**less** (const interval& *l*) const

■ $\text{self} \leq I \wedge \overline{\text{self}} \leq \overline{I}$

Returns **true** if each bound of **self** is at most the bound of *I* on the same side: **less** of IEEE 1788-2015 (Table 10.3). Two empty sets are in the relation, and an empty set and a nonempty interval are not (Table 10.4). [GAOL v5]

Example

```

1 interval e = interval::emptyset();
2 cout << interval(1,2).less(interval(1,3)) << ' '
3   << interval(1,4).less(interval(2,3)) << ' '
4   << e.less(e) << ' ' << e.less(interval(1,2));
5  true false true false

```

bool **interval::strictly_less** (const interval& *I*) const

■ $\text{self} < I \wedge \overline{\text{self}} < \overline{I}$

Returns **true** if each bound of **self** is below the bound of *I* on the same side, an infinite bound counting as below itself: **strictLess** of IEEE 1788-2015. The empty sets are as for **less**(). [GAOL v5]

Example

```

1 interval a(-GAOL_INFINITY,1), b(-GAOL_INFINITY,2);
2 cout << interval(1,2).strictly_less(interval(1,3)) << ' '
3   << a.strictly_less(b) << ' '
4   << interval(0,1).strictly_less(interval(2,3));
5  false true true

```

7.4 Relational Symbols

The relation symbols are the certainly relations, whichever build configured GAOL: the option of GAOL 4 that chose between the set, certainly and possibly relations is gone, and **configure** and **meson** refuse it. [GAOL v5] **==** and **!=** are not defined on intervals: compare them with **set_eq()**, **set_disjoint()** or **less()**.

bool **operator<** (const interval& *I1*, const interval& *I2*)

Returns *I1*.**certainly_le**(*I2*): **strictPrecedes** of IEEE 1788-2015.

bool **operator<=** (const interval& *I1*, const interval& *I2*)

Returns *I1*.**certainly_leq**(*I2*): **precedes** of IEEE 1788-2015.

bool **operator>** (const interval& *I1*, const interval& *I2*)

Returns *I1*.**certainly_ge**(*I2*).

bool **operator>=** (const interval& *I1*, const interval& *I2*)

Returns *I1*.**certainly_geq**(*I2*).

Example

```

1 cout << (interval(1,2) < interval(3,4)) << ' '
2   << (interval(1,2) <= interval(2,4)) << ' '
3   << (interval(1,3) <= interval(2,4)) << ' '
4   << (interval(3,4) > interval(1,2));
5  true true false true

```

7.5 Interval-specific relations

bool interval::straddles_zero (void) const

■ $0 \in \text{self}$

Returns **true** if **self** contains zero.

Example

```
1 interval x(0,4), y, z(-12,-5);
2
3 cout << boolalpha << x.straddles_zero() << ' '
4     << y.straddles_zero() << ' '
5     << z.straddles_zero() << endl;
6  true true false
```

Note. $I.\text{straddles_zero}() \equiv I.\text{set_contains}(0.0)$

bool interval::strictly_straddles_zero (void) const

■ $\{0\}$ interior to **self**

Returns **true** if zero is included in the interior of **self**.

Example

```
1 interval x(0,4), y, z(-12,-5);
2
3 cout << boolalpha << x.strictly_straddles_zero() << ' '
4     << y.strictly_straddles_zero() << ' '
5     << z.strictly_straddles_zero() << endl;
6  false true false
```

Note. The method call to $I.\text{strictly_straddles_zero}()$ is equivalent to $I.\text{set_strictly_contains}(0.0)$

bool interval::is_a_double (void) const

Returns **true** whenever the left and right bounds of the interval are equal: isSingleton of IEEE 1788-2015.

Example

```
1 interval x(-12.5), y;
2
3 cout << boolalpha << x.is_a_double() << ' '
4     << y.is_a_double() << ' '
5     << interval::emptyset().is_a_double() << endl;
6  true false false
```

bool interval::is_an_int (void) const

Returns **true** whenever the left and right bounds of the interval are equal and castable into an integer (type **int**).

Example

```
1 interval x(-12.0), y;
2
3 cout << boolalpha << x.is_an_int() << ' ' << y.is_an_int() << ' '
4     << interval::emptyset().is_an_int() << ' '
5     << interval(3e10).is_an_int() << endl;
6  true false false false
```

bool interval::is_canonical (void) const

Returns true if self is nonempty and contains at most two floating-point numbers.

Example

```
1 interval x(0.0), y(-14,9);
2
3 cout << boolalpha << x.is_canonical() << ' '
4     << y.is_canonical() << ' '
5     << interval::emptyset().is_canonical() << ' '
6     << interval::pi().is_canonical();
7  true false false true
```

bool interval::is_empty (void) const

■ self = $\emptyset$

Returns true if self is an empty interval: isEmpty of IEEE 1788-2015.

Example

```
1 cout << interval(4,5).is_empty() << ' '
2     << interval(5,4).is_empty() << ' '
3     << interval::emptyset().is_empty();
4  false true true
```

bool interval::is_entire (void) const

■ self = $[-\infty, +\infty]$

Returns true if self is the whole real line: isEntire of IEEE 1788-2015. [GAOL v5]

Example

```
1 cout << interval::universe().is_entire() << ' '
2     << interval::positive().is_entire() << ' '
3     << interval::emptyset().is_entire();
4  true false false
```

bool interval::is_common_interval (void) const

Returns true if self is nonempty and bounded: isCommonInterval of IEEE 1788-2015. [GAOL v5]

Example

```
1 cout << interval(1,2).is_common_interval() << ' '
2     << interval::positive().is_common_interval() << ' '
3     << interval::emptyset().is_common_interval();
4  true false false
```

bool interval::is_zero (void) const

■ self = $[0, 0]$

Returns true if self is equal to the interval containing only 0.

Example

```
1 cout << interval::zero().is_zero() << ' '
2     << interval(0.0,0.0).is_zero() << ' '
3     << interval(-0.0,+0.0).is_zero() << ' '
4     << interval(0,5).is_zero() << ' '
5     << interval::emptyset().is_zero();
6  true true true false false
```

bool interval::is_symmetric (void) const

Returns **true** if the left bound of **self** is the opposite of the right bound.
An empty interval is not symmetric.

```
Example
1 cout << interval(-5,5).is_symmetric() << ' '
2   << interval::emptyset().is_symmetric() << endl;
3  true false
```

bool interval::is_finite (void) const

Returns **true** if neither bound is infinite. The empty set, whose bounds are NaN, is finite; **is_common_interval()** is not true of it.

```
Example
1 cout << interval(4,GAOL_INFINITY).is_finite() << ' '
2   << interval::emptyset().is_finite() << ' '
3   << interval(5,80).is_finite();
4  false true true
```


8

Interval Arithmetic

The *containment principle* of (floating-point) interval arithmetic imposes that for any operation “ $\circ$ ”, and any intervals I and J , the following does hold:

$$I \circ J = \square\{i \circ j \mid i \in I, j \in J\}$$

where $\square$ is a function mapping any real set to the smallest floating-point interval containing it.

For example, if we consider the interval square root, we have:

$$\sqrt{I} = \square\{\sqrt{i} \mid i \in I\}$$

From monotonicity considerations, the square root of $[1, 2]$ is then $\sqrt{[1, 2]} = [1, \sqrt{2}]$. Now, another interpretation of the square root function is as follows:

$$\sqrt_r I = \square\{j \in \mathbb{R} \mid \exists i \in I: j^2 = i\}$$

This last definition stands for the *relational square root* and permits obtaining both negative and positive values. Hence, we have:

$$\sqrt_r[1, 2] = [-\sqrt{2}, \sqrt{2}]$$

This operator arises when we consider the relation

$$x^2 = y$$

which can alternatively be written

$$x = \sqrt_r y$$

Here, the *functional square root* is not suitable since it would induce the intersection of the domain of y with $[0, +\infty]$.

Some applications (mainly in the area of *constraint programming*) require the availability of such operators. As a consequence, GAOL offers both functional and relational versions of the main arithmetic operators.

Domains. A function is computed on the part of its argument within its domain, which is that of IEEE 1788-2015 (Tables 9.1 and 10.5): the part outside is left out, and an argument with no point of the domain gives the empty set, as $\log([-4, 0])$ does. [GAOL v5] Every function gives the empty set for an empty argument. How tight each result is, for each operation, is stated in Chapter 12.

8.1 Functional Arithmetic

8.1.1 Basic operations

```
interval interval::operator+ (void) const
interval operator+ (const interval& I, double d)
interval operator+ (double d, const interval& I)
interval operator+ (const interval& I1, const interval& I2)
```

Addition of two intervals, or of one interval and a double: `add` of IEEE 1788-2015.

```
interval interval::operator- (void) const
interval operator- (const interval& I, double d)
interval operator- (double d, const interval& I)
interval operator- (const interval& I1, const interval& I2)
```

Negation (`neg`), or subtraction (`sub`) of two intervals, or of one interval and a double.

```
interval operator* (const interval& I, double d)
interval operator* (double d, const interval& I)
interval operator* (const interval& I1, const interval& I2)
```

Multiplication of two intervals, or of one interval and a double: `mul` of IEEE 1788-2015. A zero bound times an infinite bound counts as 0, as the product of a real number of the interval by 0. [\[GAOL v5\]](#)

```
interval operator/ (const interval& I, double d)
interval operator/ (double d, const interval& I)
interval operator/ (const interval& I1, const interval& I2)
```

Functional division of two intervals, or of one interval and a double: `div` of IEEE 1788-2015. The quotient by an interval containing 0 is the hull of the quotients by its nonzero part, and the quotient by $[0, 0]$ is the empty set.

Example

```
1 cout << interval(1,2) * interval(-3,4) << ' '
2   << interval(0,1) * interval(1,GAOL_INFINITY) << ' '
3   << interval(1,2) / interval(0,1) << ' '
4   << interval(1,2) / interval(-1,1) << ' '
5   << interval(1,2) / interval::zero();
6  [-6, 8] [0, inf] [1, inf] [-inf, inf] [empty]
```

```
interval operator% (const interval& I, double d)
interval operator% (double d, const interval& I)
interval operator% (const interval& I1, const interval& I2)
```

Relational division of two intervals, or of one interval and a double (see Section 8.2, page 45).

```
interval interval::inverse (void) const
interval inverse (const interval& I)
```

■ $1/I$

Inverse of I (resp. `self`): recip of IEEE 1788-2015. The inverse of an interval containing 0 is the hull of the inverses of its nonzero part.

		Example
1	<code>cout << inverse(interval(2,4)) << ' ' << inverse(interval(-1,2));</code>	
2	 <code>[0.25, 0.5] [-inf, inf]</code>	

interval **sqr** (const interval& I)

■ I^2

Square of I : sqr of IEEE 1788-2015.

interval **sqrt** (const interval& I)

■ $\sqrt{I}$

Functional square root of I , on the part of I in $[0, +\infty]$: sqrt of IEEE 1788-2015. The bounds are the tightest ones, whatever the rounding of the square root of the C library. [\[GAOL v5\]](#)

		Example
1	<code>cout << sqrt(interval(-4,9)) << ' ' << sqrt(interval(-4,-1));</code>	
2	 <code>[0, 3] [empty]</code>	

interval **fma** (const interval& X , const interval& Y , const interval& Z)

■ $\square\{xy + z \mid x \in X, y \in Y, z \in Z\}$

Fused multiply-add: fma of IEEE 1788-2015, each bound rounded once, the tightest enclosure. [\[GAOL v5\]](#)

		Example
1	<code>cout << fma(interval(1,2), interval(3,4), interval(-1,1));</code>	
2	 <code>[2, 9]</code>	

8.1.2 Powers and roots

The following four functions are those of the namespace `gaol`; the namespace `gaol_ieee1788` has its own `pow` (Chapter 11).

interval **pow** (const interval& I , int n)

interval **pow** (const interval& I , unsigned int n)

interval **pow** (const interval& I , double p)

interval **pow** (const interval& I , const interval& J)

■ I^n
 ■ I^p
 ■ I^J

Power function. `pow` is hybrid in GAOL v5: [\[GAOL v5\]](#)

- an integer exponent takes the *integer power*, `pown` of IEEE 1788-2015: I to the power n , the negative part of I included, $x^0 = 1$ for every x , 0 included, and no value at 0 for $n < 0$; so do a double p and a degenerate interval $J = [n, n]$ whose value is an integer within the `ints`, an integer beyond them giving $[-\infty, +\infty]$;
- any other exponent takes the *real power*, `pow` of IEEE 1788-2015, on the part of I in $[0, +\infty]$, where 0^y is 0 for $y > 0$ and has no value for $y \leq 0$. An exponent $[+\infty, +\infty]$ or $[-\infty, -\infty]$, or an infinite or NaN p , gives the empty set.

The result is thus not monotone for the inclusion of J : `pow` $([-4, -1], [2, 2])$ is $[1, 16]$, and `pow` $([-4, -1], [1.5, 2.5])$ the empty set. GAOL 4 truncated a double exponent p , and computed the powers of a negative base on its magnitude.

The integer powers are computed from exact products, and are the tightest enclosures, or one double beyond where the power is within $n \cdot 2^{-104}$ of a double. The real power takes the power of CORE-MATH at the corners of the box $I \times J$ where it is the least and the greatest: the upper bound is the tightest one, and so is the lower bound but where the power at the corner is a double, which it misses by one double.

```

Example
1 cout << pow(interval(-4,-1), 2) << ' '
2   << pow(interval(-4,-1), interval(2.0)) << ' '
3   << pow(interval(-4,9), interval(0.5)) << ' '
4   << pow(interval(-4,-1), interval(1.5, 2.5)) << endl;
5  [1, 16] [1, 16] [0, 3] [empty]
6 cout << pow(interval(0.0), interval(0.0)) << ' '
7   << pow(interval(-2,2), -2) << ' '
8   << pow(interval(4), 0.5) << endl;
9  <1, 1> [0.25, inf] [1.9999999999999999, 2]
```

interval `nth_root` (const interval& I , unsigned int n)
 interval `nth_root` (const interval& I , int q)

■ ${}^n\sqrt{I}$

Computes the n th functional root of I : `rootn` of IEEE 1788-2015. For an odd n , the root of a negative number is the opposite of the root of its magnitude, and the root is defined on the whole real line; for an even n , only the part of I in $[0, +\infty]$ is considered. [\[GAOL v5\]](#) A negative q gives $1/x^{1/|q|}$, defined on $\mathbb{R} \setminus \{0\}$ for an odd q and on $]0, +\infty]$ for an even one. Returns the empty set for $n = 0$.

The bounds are the tightest ones, or one double beyond: each is proved with integer powers. The cube root is that of CORE-MATH, the tightest one, and the root of a double that is the n th power of a double is that double.

	Example
1	<code>cout << nth_root(interval(-8,27), 3) << ' '</code>
2	<code><< nth_root(interval(-4,16), 2) << ' '</code>
3	<code><< nth_root(interval(4,16), -2) << ' '</code>
4	<code><< nth_root(interval(-8,-1), -3);</code>
5	 <code>[-2, 3] [0, 4] [0.25, 0.5] [-1, -0.5]</code>

8.1.3 Exponentials and logarithms

The functions of this section, and of the next two, are bounded with CORE-MATH at the bounds of the interval, or at the points where the function reaches its extrema within it. CORE-MATH being correctly rounded, the value at a bound, computed rounding upward, is the upper bound, and the double below it the lower bound: the tightest bounds, unless the exact value is a double, which the functions give exactly where they know it happens ($\exp(0) = 1$, $\log(1) = 0$, $\sin(0) = 0$, $2^3 = 8 \dots$). [\[GAOL v5\]](#)

`interval exp (const interval& I)`

Exponential of I .

`interval exp2 (const interval& I)`

`interval exp10 (const interval& I)`

2^x and 10^x for x in I : `exp2` and `exp10` of IEEE 1788-2015. [\[GAOL v5\]](#)

`interval expm1 (const interval& I)`

`interval exp2m1 (const interval& I)`

`interval exp10m1 (const interval& I)`

$e^x - 1$, $2^x - 1$ and $10^x - 1$ for x in I , within $[-1, +\infty]$: `expm1`, `exp2m1` and `exp10m1` of IEEE 1788-2015 (Table 10.5). [\[GAOL v5\]](#)

`interval log (const interval& I)`

Natural logarithm of I , defined on $]0, +\infty]$: an interval holding no positive number gives the empty set, where GAOL 4 kept its part in $[0, +\infty]$ and gave $[-\infty, -\text{MAX}]$ for $\log([-4, 0])$. [\[GAOL v5\]](#)

`interval log2 (const interval& I)`

`interval log10 (const interval& I)`

Logarithms in base 2 and 10 of I : $\log_2$ and $\log_{10}$ of IEEE 1788-2015. [GAOL v5]

interval $\log_{1p}$ (const interval& I)
interval $\log_{2p1}$ (const interval& I)
interval $\log_{10p1}$ (const interval& I)

$\log(1+x)$, $\log_2(1+x)$ and $\log_{10}(1+x)$ for x in I , defined on $] -1, +\infty]$, $-\infty$ being their limit at -1 : $\log_{1p}$, $\log_{2p1}$ and $\log_{10p1}$ of IEEE 1788-2015 (Table 10.5). [GAOL v5]

		Example
1	cout << exp(interval(0.0)) << ' ' << log(interval(1)) << ' '	
2	<< log(interval(-4,0.0)) << ' '	
3	<< log(interval(-4,1)) << endl;	
4	<1, 1> <0, 0> [empty] [-inf, 0]	
5	cout << exp2(interval(3)) << ' ' << log2(interval(0.25)) << ' '	
6	<< log10(interval(0.0,1000)) << ' '	
7	<< expm1(interval(-1e300,0.0)) << ' '	
8	<< log1p(interval(-2,0.0)) << endl;	
9	<8, 8> <-2, -2> [-inf, 3] [-1, 0] [-inf, 0]	

8.1.4 Trigonometric functions

interval $\cos$ (const interval& I)

Returns the cosine of I , within $[-1, 1]$.

interval $\sin$ (const interval& I)

Returns the sine of I , within $[-1, 1]$.

interval $\tan$ (const interval& I)

Returns the tangent of I : $[-\infty, +\infty]$ when I holds a pole.

The three functions find the pieces of I on which they are monotonic by dividing its bounds by an enclosure of π and, where the quotients cannot tell, next to an extremum or a pole and at the large magnitudes, from the signs of the derivative at the bounds, which CORE-MATH gives exactly: their bounds are within one double of the tightest ones at every magnitude. [GAOL v5] GAOL 4 gave $[-1, 1]$ for $\cos([2^{60}, 2^{60}])$, and computed the sine as a shifted cosine, which widened narrow intervals.

		Example
1	cout << cos(interval(1,4)) << endl	
2	<< sin(interval(1,2)) << endl	
3	<< tan(interval(1,2)) << endl	
4	// 1152921504606846976 is 2 to the 60	
5	<< cos(interval(1152921504606846976.0)) << endl;	
6	[-1, 0.5403023058681398]	
7	[0.8414709848078965, 1]	
8	[-inf, inf]	
9	[-0.5567960822766418, -0.5567960822766416]	

interval acos (const interval& *I*)

Returns the arccosine of *I*, on the part of *I* in $[-1, 1]$.

interval asin (const interval& *I*)

Returns the arcsine of *I*, on the part of *I* in $[-1, 1]$.

interval atan (const interval& *I*)

Returns the arctangent of *I*.

Example

```
1 cout << acos(interval(1,3)) << ' ' << asin(interval(1)) << ' '
2   << atan(interval(1));
3  <0, 0> [1.570796326794896, 1.570796326794897]
4  [0.7853981633974482, 0.7853981633974484]
```

interval atan2 (const interval& *Y*, const interval& *X*)

Returns the hull of the angles in $(-\pi, \pi]$ of the points of ordinate in *Y* and abscissa in *X*, the point (0,0) left out, as the **atan2** of IEEE 1788-2015: **atan2**([0],[0]) is empty, and the result is $[-\pi, \pi]$ whenever the box holds points of the half-line $y = 0, x < 0$, where the angle is π , and points below it, where it is next to $-\pi$. GAOL 4 declared the function, but it raised `unavailable_feature_error`. [\[GAOL v5\]](#)

Example

```
1 interval x(1,GAOL_INFINITY);
2 cout << atan2(interval(0.0), interval(0.0)) << endl
3     << atan2(interval(-1,1), interval(-1)) << endl
4     << atan2(x, x) << endl;
5  [empty]
6  [-3.141592653589794, 3.141592653589794]
7  [0, 1.570796326794897]
```

interval sinpi (const interval& *I*)

interval cospi (const interval& *I*)

interval tanpi (const interval& *I*)

$\sin(\pi x)$, $\cos(\pi x)$ and $\tan(\pi x)$ for x in *I*: **sinPi**, **cosPi** and **tanPi** of IEEE 1788-2015 (Table 10.5). [\[GAOL v5\]](#) Their extrema and poles are at the multiples of $1/2$, which are doubles, and are found exactly, at every magnitude. **tanpi** has no value at the half-integers, which are left out: a half-integer alone gives the empty set.

interval asinpi (const interval& *I*)

interval acospi (const interval& *I*)

interval atanpi (const interval& *I*)

interval atan2pi (const interval& *Y*, const interval& *X*)

$\text{asin}(x)/\pi$, $\text{acos}(x)/\pi$, $\text{atan}(x)/\pi$ and $\text{atan2}(y,x)/\pi$: `asinPi`, `acosPi`, `atanPi` and `atan2Pi` of IEEE 1788-2015 (Table 10.5), within $[-1/2, 1/2]$, $[0, 1]$, $[-1/2, 1/2]$ and $[-1, 1]$. [\[GAOL v5\]](#)

	Example
1	<code>cout << sinpi(interval(1e17)) << ' '</code>
2	<code><< sin(interval::pi() * 1e17) << endl;</code>
3	<code> <0, 0> [-1, 1]</code>
4	<code>cout << tanpi(interval(0.5)) << ' '</code>
5	<code><< tanpi(interval(0.25,0.75)) << ' '</code>
6	<code><< asinpi(interval(1)) << ' '</code>
7	<code><< atan2pi(interval(1), interval(-1)) << endl;</code>
8	<code> [empty] [-inf, inf] <0.5, 0.5> <0.75, 0.75></code>

8.1.5 Hyperbolic functions

`interval cosh (const interval& I)`

Returns the hyperbolic cosine of I .

`interval acosh (const interval& I)`

Returns the hyperbolic arccosine of I , on the part of I in $[1, +\infty]$.

`interval sinh (const interval& I)`

Returns the hyperbolic sine of I .

`interval asinh (const interval& I)`

Returns the hyperbolic arcsine of I .

`interval tanh (const interval& I)`

Returns the hyperbolic tangent of I , within $[-1, 1]$.

`interval atanh (const interval& I)`

Returns the hyperbolic arctangent of I , on the part of I in $[-1, 1]$.

The six functions are those of CORE-MATH, where GAOL 4 took them from the mathematical library of the system, which is sometimes further than one double from the exact value. [\[GAOL v5\]](#)

	Example
1	<code>cout << cosh(interval(-1,2)) << ' '</code>
2	<code><< tanh(interval(20,30)) << endl</code>
3	<code><< acosh(interval(0,1)) << ' '</code>
4	<code><< atanh(interval(-2,2)) << endl;</code>
5	<code> [1, 3.762195691083632] [0.9999999999999998, 1]</code>
6	<code> <0, 0> [-inf, inf]</code>

8.1.6 Other functions

interval hypot (const interval& X, const interval& Y)

$\sqrt{x^2 + y^2}$ for x in X and y in Y : hypot of IEEE 1788-2015 (Table 10.5), computed at the points of the box nearest to the origin and farthest from it. [GAOL v5]

interval rsqrt (const interval& I)

$1/\sqrt{x}$ for x in I , defined on $]0, +\infty]$, $+\infty$ being its limit at 0: rSqrt of IEEE 1788-2015 (Table 10.5). [GAOL v5]

Example

```

1 cout << hypot(interval(3), interval(4)) << ' '
2   << hypot(interval(-3,1), interval(4,5)) << endl
3   << rsqrt(interval(4,16)) << ' '
4   << rsqrt(interval(0.0,4)) << endl;
5  <5, 5> [4, 5.830951894845301]
6  [0.25, 0.5] [0.5, inf]
```

Among the functions IEEE 1788-2015 recommends (Table 10.5), `compoundm1` alone is not provided, CORE-MATH having no version of it for doubles.

8.2 Relational Arithmetic

The relational functions of GAOL are the *reverse functions* of IEEE 1788-2015 (10.5.4, Table 10.1), with their arguments in another order: each computes the hull of the x of an interval I whose image by the function is in J . Their correspondence is given at the end of this section, and the namespace `gaol_ieee1788` provides them under their names (Chapter 11).

interval interval::operator% (double d) const
interval interval::operator% (const interval& I) const
interval operator% (double d, const interval& I)

■ $\text{self} \% I = \square\{z \in \mathbb{R} \mid \exists x \in \text{self}, \exists y \in I: x = yz\}$

Relational division: `div_rel(self, I,  $[-\infty, +\infty]$ )`.

Example

```

1 cout << interval(1,2) % interval(0,1) << ' '
2   << interval(1,2) % interval(-1,1);
3  [1, inf] [-inf, inf]
```

8.2.1 $(n + 1)$ -ary relational functions

Consider the relation $y = \cos x$ where x and y are interval variables. One would like to be able to express this relation in the equivalent way: $x = \text{acos } y$. However, one cannot use the `acos` function because its result is always included into the interval $[0, \pi]$. What we need here is a relational version of the `acos` function. But, since for any value x there are infinitely many values y verifying $x = \text{acos } y$, we have to take into account the domain of y . As a consequence, we define a new binary operator `acos_rel` whose definition is as follows:

$$\text{acos_rel}(Y, X) = \square\{x \in X \mid \exists y \in Y: y = \cos x\}$$

This is to be contrasted with the previous definition of the *acos* function:

$$\text{acos}(Y) = \square\{x \in \mathbb{R} \mid \exists y \in Y: x = \text{acos } y\}$$

Figure 8.1 presents the different results obtained when computing either $\text{acos}(J)$ or $\text{acos_rel}(J, I)$. An interval I that meets no piece of the preimage gives the empty set. [GAOL v5]

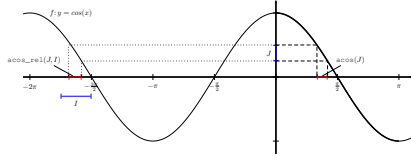


Figure 8.1: Relational cosine

interval `acos_rel` (const interval& J, const interval& I)

$$\blacksquare \text{acos_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \cos x\}$$

Returns the *relational arccosine* of J w.r.t. I .

interval `asin_rel` (const interval& J, const interval& I)

$$\blacksquare \text{asin_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \sin x\}$$

Returns the *relational arcsine* of J w.r.t. I .

interval `atan_rel` (const interval& J, const interval& I)

$$\blacksquare \text{atan_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \tan x\}$$

Returns the *relational arctangent* of J w.r.t. I .

The three functions enclose the multiples of π of the pieces of the preimage in double-double arithmetic, and add the inverse function of J before rounding the sum: their bounds are within one double of the accurate ones. [GAOL v5]

Example

```
1 cout << asin_rel(interval(0.0), interval(1,7)) << endl
2   << atan_rel(interval(1), interval(0,5)) << endl;
3 📦 [3.141592653589793, 6.283185307179588]
4 📦 [0.7853981633974482, 3.926990816987242]
```

interval `acosh_rel` (const interval& J, const interval& I)

$$\blacksquare \text{acosh_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \cosh x\}$$

Returns the *relational hyperbolic arccosine* of J w.r.t. I .

interval `asinh_rel` (const interval& J, const interval& I)

interval `atanh_rel` (const interval& J, const interval& I)

$$\blacksquare \text{asinh_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \sinh x\}$$

$$\blacksquare \text{atanh_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = \tanh x\}$$

Return the *relational hyperbolic arcsine* and *arctangent* of J w.r.t. I , that is, $\text{asinh}(J) \cap I$ and $\text{atanh}(J) \cap I$, $\sinh$ and $\tanh$ being one-to-one.

Example

```
1 cout << acosh_rel(interval(1,2), interval::universe()) << endl
2   << asinh_rel(interval(0,1), interval::universe()) << endl;
3 📦 [-1.316957896924817, 1.316957896924817]
4 📦 [0, 0.8813735870195431]
```

interval `sqr_rel` (const interval& J, const interval& I)

■ $\text{sqrt_rel}(J, I) = \square\{x \in I \mid \exists y \in J: y = x^2\}$

Returns the *relational square root* of J w.r.t. I .

Example

```
1 cout << sqrt_rel(interval(1,4), interval::universe()) << ' '
2   << sqrt_rel(interval(1,4), interval(0,10));
3  📖 [-2, 2] [1, 2]
```

interval **nth_root_rel** (const interval& J , unsigned int n , const interval& I)

■ $\text{nth_root_rel}(J, n, I) = \square\{x \in I \mid \exists y \in J: y = x^n\}$

Returns the *relational inverse n -th power* of J w.r.t. I , for $n \geq 1$. It takes the roots of `nth_root()`.

Example

```
1 interval J(1,16);
2 cout << nth_root_rel(J, 4, interval::universe()) << ' '
3   << nth_root_rel(J, 4, interval(-10,0.0));
4  📖 [-2, 2] [-2, -1]
```

interval **invabs_rel** (const interval& J , const interval& I)

■ $\text{invabs_rel}(J, I) = \square\{x \in I \mid |x| \in J\}$

Returns the *relational inverse absolute value* of J w.r.t. I . The negative part of J holds no magnitude: GAOL 4 took it as magnitudes, and gave $[-2, 2]$ where the result is empty. [GAOL v5]

Example

```
1 cout << invabs_rel(interval(1,2), interval::universe()) << ' '
2   << invabs_rel(interval(-2,-1), interval(-10,10));
3  📖 [-2, 2] [empty]
```

interval **div_rel** (const interval& K , const interval& J , const interval& I)

■ $\text{div_rel}(K, J, I) = \square\{x \in I \mid \exists z \in K, \exists y \in J: z = xy\}$

Returns the *ternary relational division* of K by J w.r.t. I , which is empty when no x of I is in the relation. [GAOL v5]

Example

```
1 cout << div_rel(interval(1,2), interval(-1,1),
2   interval(0,10)) << ' '
3   << div_rel(interval(5,6), interval(1,2),
4   interval(10,20));
5  📖 [1, 10] [empty]
```

The correspondence with the reverse functions of IEEE 1788-2015, whose last argument x is $[-\infty, +\infty]$ when it is left out:

IEEE 1788-2015	GAOL
<code>sqrRev(c, x)</code>	<code>sqrt_rel(c, x)</code>
<code>absRev(c, x)</code>	<code>invabs_rel(c, x)</code>
<code>pownRev(c, x, p), p ≥ 1</code>	<code>nth_root_rel(c, p, x)</code>
<code>sinRev, cosRev, tanRev(c, x)</code>	<code>asin_rel, acos_rel, atan_rel(c, x)</code>
<code>coshRev(c, x)</code>	<code>acosh_rel(c, x)</code>
<code>mulRev(b, c, x)</code>	<code>div_rel(c, b, x)</code>

`powRev1`, `powRev2`, `atan2Rev1`, `atan2Rev2`, `pownRev` for $p \leq 0$ and `mulRevToPair` are not provided.

8.3 Cancellative addition and subtraction

interval **cancel_minus** (const interval& X, const interval& Y)

The tightest interval Z such that $Y + Z$ contains X , $[\underline{X} - \underline{Y}, \overline{X} - \overline{Y}]$, which exists when X and Y are bounded and X is at least as wide as Y ; $[-\infty, +\infty]$ otherwise, and the empty set for an empty X and a bounded Y : **cancelMinus** of IEEE 1788-2015 (10.5.6). Whether X is at least as wide as Y is decided exactly, which rounding the differences of the bounds cannot do. [\[GAOL v5\]](#)

interval **cancel_plus** (const interval& X, const interval& Y)

cancel_minus($X, -Y$): **cancelPlus** of IEEE 1788-2015. [\[GAOL v5\]](#)

Example

```
1 cout << cancel_minus(interval(1,5), interval(0,2)) << ' '
2   << cancel_minus(interval(1,2), interval(0,3)) << ' '
3   << cancel_plus(interval(1,5), interval(0,2));
4  [1, 3] [-inf, inf] [3, 5]
```

9

Interval functions

The numeric functions of IEEE 1788-2015 (Table 10.2, 12.12.8) are those of this chapter; the namespace `gaol_ieee1788` names them as the standard does, and returns $+\infty$ and $-\infty$ for the bounds of the empty set, where `left()` and `right()` return NaN (Chapter 11).

double interval::rad (void) const

■ $\text{rad}([a, b]) = \min\{r \mid [m - r, m + r] \supseteq [a, b]\}$

Returns the radius of `self`, the smallest double r such that $[m - r, m + r]$ contains `self`, m being `self.midpoint()`: $+\infty$ for an unbounded interval, NaN for the empty set (rad of IEEE 1788-2015). [GAOL v5]

void interval::mid_rad (double& m, double& r) const

Sets m to `self.midpoint()` and r to `self.rad()` (midRad of IEEE 1788-2015). [GAOL v5]

Example

```
1 double m, r;
2 interval(1,4).mid_rad(m, r);
3 cout << m << ' ' << r << ' ' << interval(1,GAOL_INFINITY).rad();
4 2.5 1.5 inf
```

double interval::width (void) const

■ $\text{width}([a, b]) = \uparrow b - a \uparrow$

Returns the width of `self`: wid of IEEE 1788-2015. Returns NaN whenever the interval is empty, as the standard has it, where GAOL 4 returned -1 . [GAOL v5]

Example

```
1 cout << interval(4,6).width() << ' '
2   << (interval(1,next_float(1)).width()
3     == std::numeric_limits<double>::epsilon()) << ' '
4   << interval::emptyset().width();
5 2 true nan
```

double interval::mig (void) const

[6] Eldon Robert Hansen. *Global Optimization Using Interval Analysis*. Pure and Applied Mathematics. Marcel Dekker Inc., 1992.

Returns the *mignitude* of **self**: **mig** of IEEE 1788-2015. See the book by Hansen [6, chap. 3]. The mignitude of an interval $[a, b]$ is the smallest absolute value of the numbers in the interval, that is: 0 if the interval straddles 0, a if the interval is strictly positive, and $-b$ otherwise.

Note. The mignitude of the empty interval is a NaN.

Example

```

1 cout << interval(4,5).mig() << ' '
2   << interval(-6,-3).mig() << ' '
3   << interval(-3,8).mig();
4  4 3 0

```

[14] Volker Stahl. *Interval Methods for Bounding the Range of Polynomials and Solving Systems of Nonlinear Equations*. Phd. thesis, Johannes Kepler Universität, Linz, September 1995.

double interval::smig (void) const

Returns the *signed mignitude* of **self**. See Stahl's thesis [14, def. 1.3.28]. The signed mignitude of an interval $[a, b]$ is 0 if the interval straddles 0, a if the interval is strictly positive, and b otherwise.

Note. The signed mignitude of the empty interval is a NaN.

Example

```

1 cout << interval(4,5).smig() << ' '
2   << interval(-6,-3).smig() << ' '
3   << interval(-3,8).smig();
4  4 -3 0

```

double interval::mag (void) const

Returns the *magnitude* of **self**: **mag** of IEEE 1788-2015. The magnitude of an interval $[a, b]$ is the greatest absolute value of the numbers in the interval.

Note. The magnitude of the empty interval is a NaN.

Example

```

1 cout << interval(4,5).mag() << ' '
2   << interval(-6,-3).mag() << ' '
3   << interval(-10,5).mag();
4  5 6 10

```

double hausdorff (const interval& I1, const interval& I2)

Returns the Hausdorff distance between the two sets defined by intervals $I1$ and $I2$, that is:

$$\text{hausdorff}(I_1, I_2) = \max(|\underline{I_1} - \underline{I_2}|, |\overline{I_1} - \overline{I_2}|)$$

rounded upward: the tightest upper bound of the distance. [GAOL v5]

Example

```

1 cout << hausdorff(interval(4,8), interval(5,10));
2  2

```

[5] Frédéric Goualard. How do you compute the midpoint of an interval? *ACM Transactions on Mathematical Software*, 40(2), 2014.

double interval::midpoint (void) const

Returns the midpoint of `self`, mid of IEEE 1788-2015. Given a and b two finite floating-point numbers, we have the following cases, MAX being the largest positive double, `std::numeric_limits<double>::max()`:

$$\begin{cases} \text{midpoint}(\emptyset) & = \text{NaN} \\ \text{midpoint}([-\infty, +\infty]) & = 0 \\ \text{midpoint}([-\infty, b]) & = -\text{MAX} \\ \text{midpoint}([a, +\infty]) & = \text{MAX} \\ \text{midpoint}([a, b]) & = \uparrow (a+b)/2 \downarrow \end{cases}$$

The midpoint of $[a, b]$ is rounded to nearest, ties to even: it is the half of $a + b$, or the sum of the halves of a and b when $a + b$ overflows [5].

interval interval::mid (void) const

Returns the tightest interval enclosing the midpoint of `self`, which is included in `self`. With the same notations as for `midpoint()`, we have the cases:

$$\begin{cases} \text{mid}(\emptyset) & = \emptyset \\ \text{mid}([-\infty, +\infty]) & = [0, 0] \\ \text{mid}([-\infty, b]) & = [-\text{MAX}, -\text{MAX}] \\ \text{mid}([a, +\infty]) & = [\text{MAX}, \text{MAX}] \\ \text{mid}([a, b]) & = [\downarrow (a+b)/2 \downarrow, \uparrow (a+b)/2 \uparrow] \end{cases}$$

The midpoint is computed as `midpoint()` computes it, with the rounding outward. GAOL 4 did not enclose the midpoint of some intervals whose bounds are subnormal numbers. [GAOL v5]

double interval::left (void) const

$$\blacksquare \text{left}([x, y]) = x$$

Returns the left bound of `self`: NaN for the empty set, whose bounds are NaN. `gaol_ieee1788::inf()` returns $+\infty$ there, as IEEE 1788-2015 has it.

double interval::right (void) const

$$\blacksquare \text{right}([x, y]) = y$$

Returns the right bound of `self`: NaN for the empty set. `gaol_ieee1788::sup()` returns $-\infty$ there.

interval abs (const interval& I)

$$\blacksquare \text{abs}(I) = \{|x| \in \mathbb{R}^+ \mid x \in I\}$$

Returns the absolute value of I .

		Example
1	<code>cout << abs(interval(-5,6)) << ' '</code>	
2	<code><< abs(interval(-4,-2));</code>	
3	<code>📄 [0, 6] [2, 4]</code>	

double chi (const interval &I)

[12] Helmut Ratschek and Jon Rokne. Interval methods. In *Handbook of Global Optimization*, pages 751–828. Kluwer Academic, 1995.

This function, introduced by Ratscheck and Rokne [12], characterizes the degree of symmetry of intervals. Its definition is as follows:

$$\text{For } I = [a, b], \quad \text{chi}(I) = \begin{cases} -1 & \text{if } I = 0 \\ a/b & \text{if } |a| \leq |b| \\ b/a & \text{otherwise} \end{cases}$$

Example

```

1 cout << chi(interval(3,6)) << ' '
2   << chi(interval(-6,3)) << ' '
3   << chi(interval::emptyset()) << ' '
4   << chi(interval::universe()) << ' '
5   << chi(interval(-5,GAOL_INFINITY));
6  0.5 -0.5 nan 1 0

```

interval min (const interval &I, const interval &J)

■ $\min([a, b], [c, d]) = [\min(a, c), \min(b, d)]$

Returns the minimum of two intervals.

Example

```

1 cout << min(interval(5,6),interval(3,9)) << ' '
2   << min(interval::emptyset(),interval(3,8));
3  [3, 6] [empty]

```

interval max (const interval &I, const interval &J)

■ $\max([a, b], [c, d]) = [\max(a, c), \max(b, d)]$

Returns the maximum of two intervals.

Example

```

1 cout << max(interval(5,6),interval(3,9)) << ' '
2   << max(interval::emptyset(),interval(3,8));
3  [5, 9] [empty]

```

9.1 Integer functions

The functions of this section are those of IEEE 1788-2015 (Table 9.1): each is non-decreasing, so that the bounds of the result are its values at the bounds of the interval, and each is exact. None of them depends on the rounding direction.

interval floor (const interval &I)

■ $[\text{floor}(I.\text{left}()), \text{floor}(I.\text{right}())]$

Example

```

1 cout << floor(interval(4.5,6.5)) << ' '
2   << floor(textToInterval("[-10.4,3.5]"));
3  [4, 6] [-11, 3]

```

interval ceil (const interval &I)

■ $[\text{ceil}(I.\text{left}()), \text{ceil}(I.\text{right}())]$

Example

```

1 cout << ceil(interval(4.5,6.5)) << ' '
2   << ceil(textToInterval("[-10.4,3.5]"));
3  [5, 7] [-10, 4]

```

interval integer (const interval &*I*)

■ `[ceil(I.left()), floor(I.right())]`

Narrows down the bounds to the closest integers: the empty set when *I* holds no integer. Note that the resulting bounds are still **double** numbers, and may therefore not be representable with integral types.

Example

```
1 cout << integer(interval(4.5,6.5)) << ' '
2   << integer(interval(4.2,4.8));
3  [5, 6] [empty]
```

interval sign (const interval &*I*)

The signs of the elements of *I*: -1 below 0, 0 at 0, 1 above (sign). [\[GAOL v5\]](#)

interval trunc (const interval &*I*)

The elements of *I* rounded toward zero (trunc). [\[GAOL v5\]](#)

interval round_ties_to_even (const interval &*I*)

interval round_ties_to_away (const interval &*I*)

The elements of *I* rounded to the nearest integer, halfway values to the even one, respectively away from zero (roundTiesToEven, roundTiesToAway). [\[GAOL v5\]](#)

Example

```
1 cout << sign(interval(-2,3)) << ' '
2   << trunc(interval(-2.5,3.7)) << ' '
3   << round_ties_to_even(interval(0.5,2.5)) << ' '
4   << round_ties_to_away(interval(0.5,2.5));
5  [-1, 1] [-2, 3] [0, 2] [1, 3]
```

9.2 Splitting methods

void interval::split (interval& *I1*, interval& *I2*) const

Splits **self** into two parts using `midpoint()`; returns the left part in *I1* and the right part in *I2*.

I1 or *I2* may be equal to **self**.

Example

```
1 interval I1a, I2a,
2   I3(1.0,next_float(1.0)),
3   I1b, I2b;
4 interval(4,5).split(I1a,I2a);
5 I3.split(I1b,I2b);
6 cout << I1a << ' ' << I2a << ' '
7   << I1b.set_eq(interval(1.0)) << ' ' << I2b.set_eq(I3) << endl;
8  [4, 4.5] [4.5, 5] true true
```

interval interval::split_left (void) const

■ `split_left([a, b]) = [a, m]`

Splits `self` into two parts using `m = midpoint()` and returns the left part.

Example

```
1 cout << interval(4,5).split_left();  
2  [4, 4.5]
```

`interval interval::split_right (void) const`

■ `split_right([a, b]) = [m, b]`

Splits `self` into two parts using `m = midpoint()` and returns the right part.

Example

```
1 cout << interval(4,5).split_right();  
2  [4.5, 5]
```

9.3 Union and intersection

`interval operator& (const interval& I1, const interval& I2)`

■ $I1 \cap I2$

Returns the interval resulting from the intersection of $I1$ and $I2$: intersection of IEEE 1788-2015. The intersection of disjoint intervals is the empty set `interval::emptyset()`, whose bounds are NaN. GAOL 4 kept the greatest left bound and the smallest right bound, $[3, 2]$ for $[1, 2] \cap [3, 4]$, which the operations computing on the bounds did not take for empty. [\[GAOL v5\]](#)

Example

```
1 cout << (interval(4,6) & interval(5,9)) << ' '  
2 << (interval(1,2) & interval(3,4));  
3  [5, 6] [empty]
```

`interval operator| (const interval& I1, const interval& I2)`

■ $\square(I1 \cup I2)$

Returns the hull of the union of $I1$ and $I2$: `convexHull` of IEEE 1788-2015.

Example

```
1 cout << (interval(3,6) | interval(9,12));  
2  [3, 12]
```

10

Input/output

10.1 Reading intervals

`istream& operator>> (istream& in, interval& I)`

Reads a line from the input stream *in*, and assigns to *I* the interval it writes. If the line is syntactically ill-formed, *I* is set to the empty set, and an `input_format_error` exception is thrown (see Section 17.1, page 80) if the library was compiled with exceptions enabled (see Section 2.5, page 7); alternatively, it prints an error message to `cerr` and aborts if exceptions were disabled. An ill-formed line, a blank one included, also sets the `failbit` of *in*, which `in.clear()` resets before reading on; a function called with an argument it does not take throws `invalid_action_error` instead. Where no line is left, *in* gets its `failbit`, *I* is left as it was, and nothing is thrown, as for a number: `while (in >> x)` reads the lines one after the other and ends with the input. [GAOL v5] GAOL 4 read the empty text left, and threw `input_format_error`.

Example

```
1 std::istringstream in("[1, 2]\n[0.1, 0.2]\n");
2 interval a, b;
3 in >> a >> b;
4 cout << a << ' ' << b;
5  [1, 2] [0.09999999999999999, 0.20000000000000001]
```

10.1.1 Input format

A string to be translated into an interval must have the following syntax (with terminals in lower case and non-terminals in slanted upper case):

EXPR

```
: NUMBER
| UNCERTAIN_NUMBER           // An interval literal, never a bound
| LITERAL
| dmin                       // Smallest positive normal floating-point number
| dmax                       // Largest positive floating-point number
| pi
| inf                       // Floating-point positive “infinity”
```

```

| infinity // The same
| empty // Empty interval
|  $EXPR + EXPR$ 
|  $EXPR - EXPR$ 
|  $EXPR * EXPR$ 
|  $EXPR / EXPR$ 
|  $- EXPR$ 
|  $+ EXPR$ 
| FUNCTION_CALL
| ( EXPR )
;

LITERAL
: [ EXPR ]
| [ EXPR , EXPR ]
| [ EXPR , ] // Right bound  $+\infty$ 
| [ , EXPR ] // Left bound  $-\infty$ 
| [ , ] // Entire real line
| [ entire ] // Entire real line
| [ ] // Empty interval
| < EXPR , EXPR >
;

FUNCTION_CALL
: NAME ( EXPR )
| NAME ( EXPR , EXPR )
| NAME ( EXPR , EXPR , EXPR )
;

```

Every string is an expression, computed with interval arithmetic: the literals of intervals, the numbers, the constants and the functions combine with the operators in any order, and a literal stands wherever a number may, in a bound too. [GAOL v5] $1+[1,2]$, $2*\cos([0,1])$ and $[\cos([0,1]), 2]$ are read. GAOL 4 read a string with two grammars, one for the intervals between brackets and one for the numbers, and refused $1+[1,2]$ and $[1,2]+1+[1,2]$ while it read $[1,2]*2$. [empty] is the form [*EXPR*] of the expression *empty*.

Spaces, tabs and newlines are not significant except in numbers, and the case of letters is ignored, as IEEE 1788-2015 has it for its literals: [Empty], [1, Inf], SIN(1) and 1E3 are read. [GAOL v5] The “+” sign before numbers and *inf* is optional.

Numbers. A *NUMBER* is written in decimal, with an optional point and exponent (12, 0.1, 1e-3), or in hexadecimal, with a binary exponent (0x1.3p-1). Numbers are read exactly, whatever their number of digits: a number that is a double is read as that double, and any other one is replaced by the tightest interval of doubles containing it. [GAOL v5] GAOL 4 read them with the *strtod()* of the C library, which did not enclose 0.1 where it rounds to nearest in every rounding direction (the C runtime of Windows, must on 64-bit ARM processors). The point of a number is a point whatever the locale of the program, a locale writing a decimal comma included (as *setlocale(LC_ALL, "")* sets under a French one). [GAOL v5] GAOL 4 read 0.1 as 0 there, *strtod()* stopping at the point.

The uncertain form. An *UNCERTAIN_NUMBER* of IEEE 1788-2015 (12.11.3) is a decimal number followed by ?, a radius in units of its last digit (half a unit when it is left out, and an infinite radius for a second ?), an optional direction, u or d, which keeps only the radius above or below, and an

optional exponent: `3.56?1` is `[3.55, 3.57]`, `3.56?1e2` is `[355, 357]`, and `-10??u` is `[-10, +∞]`. [GAOL v5] It is an interval literal of its own, not a bound: `"[5?1]"` and `"[1, 3.56?1]"` throw `input_format_error`, as IEEE 1788-2015 has it (12.11.4), while `"3.56?1*2"` is `[7.1, 7.14]`.

Functions. A *NAME* is the name of a function of GAOL, with its arguments, whatever the case of its letters: `gaol::textToInterval()`, the constructor from two strings and `operator>>` read the names of GAOL, and `gaol_ieee1788::textToInterval()` those of IEEE 1788-2015 (see Section 11.3, page 66), as each namespace has its own `pow`. [GAOL v5] The names of GAOL are:

```
sqr, sqrt, cbrt, pow, nth_root, exp, exp2, exp10, expm1, exp2m1, exp10m1,
log, log2, log10, log1p, log2p1, log10p1, sin, cos, tan, asin, acos, atan,
atan2, sinh, cosh, tanh, asinh, acosh, atanh, sinpi, cospi, tanpi, asinpi,
acospi, atanpi, atan2pi, hypot, rsqrt, sign, ceil, floor, trunc, integer,
round_ties_to_even, round_ties_to_away, abs, min, max, inverse, fma
```

Each is the function of GAOL of that name (Chapters 8 and 9); `pow` is the `pow` of the namespace `gaol` (see Section 8.1.2, page 39), and `cbrt(x)` is `nth_root(x,3)`. A name that is not in the list, or a call with a wrong number of arguments, throws `input_format_error`. The second argument of `nth_root` shall be a point interval that can be evaluated as an integer, negative ones included; otherwise, an `invalid_action_error` exception is thrown (or an error is reported, depending on whether exceptions were enabled or not at configuration time—see Section 2.5, page 7). GAOL 4 read the trigonometric and hyperbolic functions, `exp`, `log`, `pow`, `sqrt` and `nth_root` only.

Bounds. The notations `"n"` and `"[n]"` are equivalent. Expressions in bounds are evaluated using interval arithmetic, intervals included; the left (resp. right) bound of the result is then used, depending on the side it appeared in: `"[cos([0,1]), 2]"` is `[cos 1, 2]`. A bound left out is infinite. The two bounds in the string `"<a, b>"` must be expressions that evaluate to the same double even for different rounding directions: an `input_format_error` exception is raised otherwise, as for `"<0.1, 0.1>"`.

Infinites. An interval literal whose left bound is $+\infty$ or whose right bound is $-\infty$ is the empty set, as `numsToInterval` of IEEE 1788-2015 has it: `"[inf]"` and `"[-inf, -inf]"` are empty. [GAOL v5] The expression `"inf"` alone is still `[MAX, +∞]`, and `"-inf"` `[-∞, -MAX]` (see Section 5.1, page 21).

Threads. Several threads may read strings at once, in parallel: the lexer is reentrant and the parser pure, and each reading of a string has a state of its own. [GAOL v5] GAOL 4 kept that state in global variables, and crashed there.

Errors. A string that is not an expression throws `input_format_error`. An error found anywhere in the string stops the reading, and no interval is given: `"[nth_root(8,1.5)]+[1,2]"` throws, where the two grammars of GAOL 4 gave `[-∞, +∞]`. [GAOL v5]

	Example
1	<code>interval x = textToInterval("[4, 6*7]");</code>
2	<code>interval y = textToInterval("[-inf, dmax]");</code>
3	<code>interval z("[3.14,3.15]/8", "[3.14,3.15]/7");</code>
4	<code>interval t = textToInterval("[3.14,3.15]/[7,8]");</code>
5	<code>cout << textToInterval("[1,]") << ' ' << textToInterval("[]") << ' '</code>

```

6      << textToInterval("[Entire]") << endl;
7  📄 [1, inf] [empty] [-inf, inf]
8  cout << textToInterval("[0x1.3p-1, 2/3]") << ' '
9      << textToInterval("3.56?1") << endl;
10 📄 [0.59375, 0.6666666666666668] [3.549999999999999, 3.570000000000001]
11 cout << textToInterval("1+[1,2]") << ' '
12      << textToInterval("[cos([0,1]), 2]") << endl;
13 📄 [2, 3] [0.5403023058681396, 2]

```

10.2 Writing intervals

Intervals may be printed into a stream like any other C++ primitive type by using the “<<” operator.

ostream& operator<< (ostream& out, const interval& I)

Prints the interval I to the output stream out . The way the intervals are actually displayed depends on the active format (see next section). However, whatever the format, an empty interval is always displayed as [empty]

In the decimal formats, each bound is rounded outward, whatever the C library: its magnitude is written by the C library, rounded to nearest, then compared exactly with the bound, and its last digit is moved by one when it is on the wrong side. The text read back therefore encloses the interval written. [GAOL v5] GAOL 4 let the C library round each bound in the direction it set, which the C runtime of Windows and musl do not honour.

std::string exact_string (const interval& I)

Returns what **operator<<** writes in the format **hexa** (see next section), without reading nor changing the global output format: “[empty]” for the empty set, and otherwise the two bounds in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which **textToInterval()** reads back bit for bit. This is the exact text representation of the standard, its **intervalToExact**. [GAOL v5] The text is the same whatever the locale of the program, a locale writing a decimal comma included, and whatever the C library.

Example

```

1  cout << exact_string(textToInterval("0.1"));
2  📄 [0x1.9999999999999p-4, 0x1.9999999999999ap-4]

```

10.2.1 Converting intervals to strings

For convenience, the **interval** class provides a conversion operator into the standard C++ type **string**, which writes the interval in the active format.

Example

```

1  interval I(3,4);
2  string s = "test line embedding " + string(I) + " as a string";
3  // Now, s is "test line embedding [3, 4] as a string"

```

10.2.2 Output format

Intervals may be displayed following five different formats:

agreeing. By printing all the digits that are the same in the left and right bounds followed by an interval containing the remaining digits:

“3.141~[5926, 6001]” stands for “[3.1415926, 3.1416001]”

Note that if the bounds do not have any agreeing digit, there will still be a tilde before the bracketed part. The digits of each bound are kept: GAOL 4 dropped from both bounds what followed the last character of the left one that is not a zero, and wrote [1.25, 1.2567] 1.25~[,]. [\[GAOL v5\]](#)

Example

```
1 interval::format(interval_format::agreeing);
2 std::cout << interval(4,6) << ' '
3   << interval(1.25,1.2567) << std::endl;
4 📦 ~[4., 6.] 1.25~[0, 67]
```

bounds. By printing the bounds between square brackets. Degenerate intervals whose left and right bound are equal are printed with angles:

Example

```
1 interval::format(interval_format::bounds);
2 std::cout << interval(3,5) << ' ' << interval(4) << std::endl;
3 📦 [3, 5] <4, 4>
```

A degenerate interval whose value has more digits than those displayed is printed with its two bounds rounded outward: `interval(0.1)` is printed `<0.1, 0.10000000000000001>`.

width. by printing their midpoint and their width:

Example

```
1 interval::format(interval_format::width);
2 std::cout << interval(4,5) << std::endl;
3 📦 4.5 (+/- 0.5)
```

center. by printing their midpoint only.

hexa. by printing their left and right bounds in the hexadecimal-significand form of IEEE 1788-2015 (13.4.1), which is exact and which `textToInterval()` reads back bit for bit, without the round-off error of a binary-to-decimal conversion: the recovery requirement of the standard (13.4). GAOL 4 printed the sixteen hexadecimal digits of each double, which its parser did not read. [\[GAOL v5\]](#)

Example

```
1 interval::format(interval_format::hexa);
2 std::cout << textToInterval("0.1") << std::endl;
3 📦 [0x1.999999999999p-4, 0x1.999999999999ap-4]
```

Note that the **bounds** and **hexa** formats are the ones read back as an input (see previous section).

The choice of the format to use is made through the following static methods:

```
void interval::format (interval_format::format_t f)          static
interval_format::format_t interval::format (void)            static
```

The first form of the method allows modifying the format to use in subsequent printing of intervals. The second form reports what is the current form in use. It returns a value of type `interval_format::format_t` (see below and 10.2.4 for an example of use). The format is global to the program: `exact_string()` gives the hexadecimal text without changing it.

interval_format

struct

Structure type used to choose the output format for intervals. It has five possible values of type `interval_format::format_t`:

- `interval_format::agreeing.`
- `interval_format::bounds.`
- `interval_format::width.`
- `interval_format::center.`
- `interval_format::hexa.`

Example

```

1  interval I(interval::pi());
2
3  interval::format(interval_format::agreeing);
4  cout << I << "\n";
5  // Prints 3.14159265358979-[3, 4]
6  // The ~[] part is dropped if the bounds agree on all digits
7
8  interval::format(interval_format::bounds);
9  cout << I << "\n";
10 // Prints [3.141592653589793, 3.141592653589794]
11
12 interval::format(interval_format::width);
13 cout << I << "\n";
14 // Prints 3.141592653589793 (+/- 2.220446049250313e-16)
15
16 interval::format(interval_format::center);
17 cout << I << "\n";
18 // Prints 3.141592653589793
19
20 interval::format(interval_format::hexa);
21 cout << I << "\n";
22 // Prints [0x1.921fb54442d18p+1, 0x1.921fb54442d19p+1]
```

10.2.3 Choosing the number of digits to display

You can manipulate the number of digits to print by using the `precision()` static methods of the interval class. The precision of the stream, which `std::setprecision` sets, is not used for intervals, and is left as it was: the numbers written after an interval keep it. The width of the stream (`std::setw`) and its adjustment apply to the whole interval, which `std::internal` pads as `std::right` does. [GAOL v5] GAOL 4 set the precision of the stream to that of the intervals and left it so, and `std::setw` padded the [of an interval alone.

`std::streamsize interval::precision (void)`

static

Returns the current number of digits used for printing bounds of intervals, 16 by default.
See example below.

`std::streamsize interval::precision (std::streamsize n)` `static`

Set the number of digits to use for printing bounds to *n*. In addition, returns the number of digits previously used.
See example below.

10.2.4 Example

Example

```
1 #include <iostream>
2 #include <gaol/gaol.h>
3 using namespace gaol;
4
5 using std::cout;
6 using std::endl;
7
8 int main(void)
9 {
10     interval::precision(4);
11     interval::format(interval_format::bounds);
12     cout << interval::pi() << endl;
13
14     if (interval::format() != interval_format::bounds) {
15         cout << interval::pi() << endl;
16     } else {
17         interval::precision(16);
18         interval::format(interval_format::width);
19         cout << interval::pi() << endl;
20     }
21     gaol::cleanup();
22     return 0;
23 }
```

The previous program has the following output:

Output

```
[3.141, 3.142]
3.141592653589793 (+/- 2.220446049250313e-16)
```

The first call to `interval::format()` is unnecessary since the default format is `interval_format::bounds`.

Note. An interval written in decimal and read back gives an interval that encloses it, whatever the precision chosen, the bounds being rounded outward; the fewer the digits, the wider. The format `hexa` gives the same interval back, bit for bit.

The `interval::pi()` constant is a predefined *canonical* interval containing π (see Section 6, page 25). Here, the width of the interval is equal to the ϵ of the format.

The `interval_format::width` format may be useful whenever the number of digits displayed is insufficient to know whether the result is a single floating-point number or an interval whose size is very small (consider for example the

first result above), because *we have the guarantee that if the actual width of an interval is greater than zero, the width displayed will also be different from zero.* Another indication is that a degenerate interval is displayed between angles.

11

The names of IEEE 1788-2015

GAOL names its operations its own way: the reverse function `coshRev( $c, x$ )` of IEEE 1788-2015 is `acosh_rel( $c, x$ )`, `mulRev( $b, c, x$ )` is `div_rel( $c, b, x$ )`, with its arguments in another order, and `roundTiesToEven( $x$ )` is `round_ties_to_even( $x$ )`. The namespace `gaol_ieee1788`, which `gaol/gaol.h` brings along, gives each operation of the standard that GAOL provides the name and the argument order the standard gives it, so that a reader of the standard finds it without looking for its translation. It is not in `gaol`, and it holds the type `interval` as well: one directive lets a program use GAOL under the names of the standard. [\[GAOL v5\]](#)

Example

```
1 #include <iostream>
2 #include <gaol/gaol.h>
3 using namespace gaol_ieee1788;
4
5 int main(void)
6 {
7     interval x = textToInterval("[1, 2]");
8     interval y = mulRev(numsToInterval(2, 2), x);    // x / 2
9     interval z = sinPi(x) + rootn(y, 3);
10    bool b = strictLess(x, entire());
11    std::cout << std::boolalpha
12              << x << ' ' << y << ' ' << z << ' ' << b << std::endl;
13    gaol::cleanup();
14    return 0;
15 }
16  [1, 2] [0.5, 1] [-0.2062994740159004, 1] false
```

A program opens `gaol` or `gaol_ieee1788`, not both (see Section 3.3, page 14). A name of the program's own that one of the standard shadows, a constant `inf` for instance, is to be qualified: `gaol_ieee1788::inf(x)`. So is `less(x, y)` next to `using namespace std;`, where it is ambiguous with the class template `std::less`.

11.1 Where the standard and GAOL differ

The functions of GAOL that have the name and the meaning the standard gives them (`sin`, `exp`, `sqrt`, `min`...) are the same functions in `gaol_ieee1788`. Where the standard and GAOL differ, the names of `gaol_ieee1788` follow the standard:

- `pow(x, y)` is the `pow` of Table 9.1, defined for $x > 0$, and for $x = 0$ when $y > 0$, where the `pow` of `gaol` takes the integer power for an integer exponent, a negative base included (see Section 8.1.2, page 39). `pow(x, p)` with a number p is `pow(x, [p])`, `pow(x, 2)` included: the power with an integer exponent is `pown(x, n)` in the standard, whose n is an integer rather than an interval. An integer exponent beyond the `ints` gives the power of CORE-MATH at the bounds of x , where the `pow` of `gaol` gives $[-\infty, +\infty]$;
- `inf(x)` and `sup(x)` are $+\infty$ and $-\infty$ for the empty set (Table 10.2), where `left()` and `right()` are NaN, and `inf` returns -0 for a left bound 0 (12.12.8);
- `isMember(m, x)` is false for an infinite m (10.6.3);
- `textToInterval` and `exactToInterval` read the names of the functions of the standard, and return the empty set for a string that is no interval (12.1.3), where `gaol::textToInterval` reads the names of GAOL and throws (see Section 11.3, page 66);
- `pownRev(c, x, p)` throws `std::invalid_argument` for $p \leq 0$, which GAOL does not provide, rather than give an interval it has not computed.

Example

```

1 cout << pow(interval(-4,-1), interval(2.0)) << ' '
2   << pown(interval(-4,-1), 2) << ' '
3   << pow(interval(2,3), interval(1e10)) << endl;
4  [empty] [1, 16] [1.797693134862315e+308, inf]
5 cout << inf(empty()) << ' ' << sup(empty()) << ' '
6   << isMember(GAOL_INFINITY, entire()) << ' '
7   << textToInterval("[1, 2]") << endl;
8  inf -inf false [empty]
```

11.2 The operations

The last argument x of the reverse functions is optional, and $[-\infty, +\infty]$ when it is left out (10.5.4).

<code>gaol_ieee1788</code>	GAOL
<i>Constants and constructors (10.5.2, 10.5.8)</i>	
<code>empty()</code> , <code>entire()</code>	<code>interval::emptyset()</code> , <code>interval::universe()</code>
<code>numsToInterval(l, u)</code>	<code>interval(l, u)</code>
<code>textToInterval(s)</code>	<code>gaol::textToInterval(s)</code> with the names of the standard, the empty set for a wrong string

gaol_ieee1788	GAOL
<i>Forward elementary functions (Table 9.1)</i>	
neg, add, sub, mul, div	-x, x + y, x - y, x * y, x / y
recip(x)	inverse(x)
sqr, sqrt, fma	the same
pown(x, p)	gaol::pow(x, p) for an int p
pow(x, y), pow(x, p)	the pow of the standard
exp, exp2, exp10, log, log2, log10	the same
sin, cos, tan, asin, acos, atan, atan2	the same
sinh, cosh, tanh, asinh, acosh, atanh	the same
sign, ceil, floor, trunc	the same
roundTiesToEven, roundTiesToAway	round_ties_to_even, round_ties_to_away
abs, min, max	the same
<i>Recommended functions (Table 10.5)</i>	
rootn(x, q)	nth_root(x, q)
expm1, exp2m1, exp10m1	the same
logp1, log2p1, log10p1	log1p, log2p1, log10p1
hypot, rSqrt	hypot, rsqrt
sinPi, cosPi, tanPi	sinpi, cospi, tanpi
asinPi, acosPi, atanPi, atan2Pi	asinpi, acospi, atanpi, atan2pi
<i>Reverse functions (Table 10.1)</i>	
sqrRev(c, x), absRev(c, x)	sqr_rel(c, x), invabs_rel(c, x)
pownRev(c, x, p), $p \geq 1$	nth_root_rel(c, p, x)
sinRev, cosRev, tanRev(c, x)	asin_rel, acos_rel, atan_rel(c, x)
coshRev(c, x)	acosh_rel(c, x)
sinhRev, tanhRev(c, x) (not in the standard, named after coshRev)	asinh_rel, atanh_rel(c, x)
mulRev(b, c, x)	div_rel(c, b, x)
<i>Cancellative operations and set operations (10.5.6, 10.5.7)</i>	
cancelMinus, cancelPlus	cancel_minus, cancel_plus
intersection(x, y), convexHull(x, y)	x & y, x y
<i>Numeric functions (Table 10.2)</i>	
inf(x), sup(x)	x.left(), x.right(), but for the empty set
mid, wid, rad, mag, mig(x)	x.midpoint(), width(), rad(), mag(), mig()
midRad(x, m, r)	x.mid_rad(m, r)
<i>Boolean functions (10.5.10, Tables 10.3 and 10.4)</i>	
isEmpty(x), isEntire(x)	x.is_empty(), x.is_entire()
equal(a, b), subset(a, b)	a.set_eq(b), a.set_leq(b)
interior(a, b), disjoint(a, b)	a.set_le(b), a.set_disjoint(b)
less(a, b), strictLess(a, b)	a.less(b), a.strictly_less(b)
precedes(a, b), strictPrecedes(a, b)	a.certainly_leq(b), a.certainly_le(b)
isCommonInterval(x), isSingleton(x)	x.is_common_interval(), x.is_a_double()
isMember(m, x)	x.set_contains(m), false for an infinite m
<i>Input and output (Clause 13)</i>	
intervalToText(x)	operator<< in the current format

gaol_ieee1788	GAOL
<code>intervalToExact(x)</code>	<code>exact_string(x)</code>
<code>exactToInterval(s)</code>	<code>textToInterval(s)</code>
<i>Expressions</i>	
<code>pown(e, n), pow(e1, e2)</code>	the expressions of <code>pown</code> and of the <code>pow</code> of the standard

11.3 Reading with the names of the standard

`textToInterval(s)` reads a string as `gaol::textToInterval()` does (see Section 10.1.1, page 55), with the names of the functions of IEEE 1788-2015: those of Tables 9.1 and 10.5 that GAOL provides, whatever the case of their letters. [GAOL v5]

`neg, add, sub, mul, div, recip, sqr, sqrt, fma, pown, pow, exp, exp2, exp10, log, log2, log10, sin, cos, tan, asin, acos, atan, atan2, sinh, cosh, tanh, asinh, acosh, atanh, sign, ceil, floor, trunc, roundTiesToEven, roundTiesToAway, abs, min, max, rootn, expm1, exp2m1, exp10m1, logp1, log2p1, log10p1, hypot, rSqrt, sinPi, cosPi, tanPi, asinPi, acosPi, atanPi, atan2Pi`

`pow` is the `pow` of Table 9.1, and the second argument of `pown` and `rootn` is an integer. The names of GAOL alone (`nth_root`, `cbrt`, `inverse`, `integer`, `log1p`, `round_ties_to_even...`) are no functions there, and give the empty set, as any string that is no interval. As `pow`, the function `textToInterval` is in the namespace `gaol` too, where it reads the names of GAOL and throws `input_format_error` for a wrong string: a program calls the one of the namespace it opens.

Example

```

1 cout << textToInterval("pown([2,5],5)") << ' '
2   << textToInterval("pow([-4,-1],2)") << ' '
3   << textToInterval("nth_root(8,3)") << endl;
4  [32, 3125] [empty] [empty]
```

With `using namespace gaol;`, `textToInterval("pow([-4,-1],2)")` is `[1, 16]`, and `textToInterval("pown([2,5],5)")` throws.

11.4 What is not provided

GAOL has no decorations: only bare intervals are provided (Clause 11). Nor does it provide `mulRevToPair` (10.5.5), `powRev1`, `powRev2`, `atan2Rev1` and `atan2Rev2` (Table 10.1), `pownRev` for $p \leq 0$, `compoundm1` (Table 10.5), the slope functions (Table 10.6), `overlap` (10.6.4), the reduction operations `sum`, `dot`, `sumSquare` and `sumAbs` (12.12.12), and the exact operations of 12.13.5. The end of `gaol/gaol_ieee1788.h` keeps this list.

12

Accuracy of the operations

IEEE 1788-2015 requires an implementation to document the tightness of each of its interval operations (12.10.3). This chapter does so for `interval`, from the algorithm of each operation; the tests of `tests` check each statement on random and special arguments (see Section 2.7, page 8), and the page `doc/accuracy.md` gives, for each, the algorithm and the largest distance the tests found. [\[GAOL v5\]](#)

12.1 Accuracy modes

IEEE 1788-2015 defines three accuracy modes (12.10.1), for an operation f , its exact value f^* , and an input box x :

- *tightest*: $f(x)$ is the hull, in doubles, of $f^*(x)$: each bound is the nearest double on its side of the exact bound;
- *accurate*: $f(x)$ encloses $f^*(x)$, and is within

$$\text{nextOut}(f_{\text{tightest}}(\text{nextOut}(x))),$$

`nextOut` moving each bound one double outward: roughly, $f(x)$ is no wider than the tightest result for an input one double wider, plus one double on each side;

- *valid*: $f(x)$ encloses $f^*(x)$.

The standard requires the basic operations, the integer functions and `abs`, `min` and `max` to be tightest; the other elementary functions, the reverse functions and the recommended functions to be valid, and recommends them to be accurate (12.10.2). Below, “within k doubles” means that each bound is at most k doubles beyond the tightest one.

GAOL bounds its elementary functions with CORE-MATH, which is correctly rounded in the rounding direction in effect: computing upward, GAOL takes the value at a bound as the upper bound, with nothing to add, and the double below it as the lower one. Those are the tightest bounds, the second one unless the exact value is a double, which the operations give exactly where it happens. Where an algorithm evaluates the function at the bounds of the interval, the bounds of the interval are therefore the tightest ones.

The statements hold for GAOL as the three builds make it, with the code including its headers compiled with the flags of interval arithmetic (see Section 3.1, page 12), for the SSE2 and the FPU intervals, whatever the rounding direction the calling code left, and on every architecture and with every compiler alike. The intervals of floats are not covered.

12.2 Tightness of each operation

Operations	Tightness
<i>Basic operations (Table 9.1): tightest required</i>	
<code>-x, +, -, *, /, inverse, sqr</code>	tightest
<code>sqr</code>	tightest, whatever the rounding of the square root of the C library
<code>fma</code>	tightest
<i>Integer and absmax functions: tightest required</i>	
<code>floor, ceil, integer, sign, trunc, round_ties_to_even, round_ties_to_away, abs, min, max</code>	tightest, and exact, in every rounding direction
<i>Powers, exponentials and logarithms: valid required, accurate recommended</i>	
<code>pow(x, n)</code> , integer n (pown)	$n = 0, 1, 2$: tightest. $n \geq 3$: the tightest bounds, or one double beyond where the power is within $n \cdot 2^{-104}$ of a double; exact where the power is a double; within about $2(n-1)$ doubles below 2^{-968} and at the overflow. Negative n : valid, within 2 doubles where x^n is normal
<code>pow(x, y)</code> (pow)	tightest at the corners of the box with finite bounds, but for a lower bound one double below where the power at the corner is a double; valid with infinite bounds
<code>exp, exp2, exp10, log, log2, log10</code>	tightest; exact where the value is a double ($\exp(0)$, $\log(1)$, 2^n , 10^n for $0 \leq n \leq 22 \dots$)
<i>Trigonometric and hyperbolic functions: valid required, accurate recommended</i>	
<code>cos, sin</code>	tightest where the function is evaluated at the bounds, at every magnitude; -1 and 1 exactly at the extrema; $[-1, 1]$ from the width 2π on, π being the double below π
<code>tan</code>	tightest between two poles, at every magnitude; $[-\infty, +\infty]$ exactly when a pole is within x , but for the widths between π and π
<code>asin, acos, atan, atan2</code>	tightest; exactly 0, or the tightest enclosures of $\pm\pi/4$, $\pm\pi/2$ and π , where the value is one of them
<code>sinh, cosh, tanh, asinh, acosh, atanh</code>	tightest
<i>Recommended functions (Table 10.5)</i>	
<code>nth_root(x, n)</code> , $n > 0$	$n = 1, 2, 3$: tightest. Otherwise accurate: the tightest bounds or one double beyond, proved with integer powers
<code>nth_root(x, q)</code> , $q < 0$	accurate: the root above, then the tightest division
<code>expm1, exp2m1, exp10m1, log1p, log2p1, log10p1, hypot, rsqrt</code>	tightest

Operations	Tightness
<code>sinpi</code> , <code>cospi</code> , <code>tanpi</code> , <code>asinpi</code> , <code>acospi</code> , <code>atanpi</code> , <code>atan2pi</code>	tightest, at every magnitude
<i>Reverse functions (Table 10.1): valid required, accurate recommended</i>	
<code>sqrt_rel</code> , <code>nth_root_rel</code> ,	accurate
<code>acosh_rel</code> , <code>asinh_rel</code> , <code>atanh_rel</code>	
<code>invabs_rel</code>	tightest
<code>asin_rel</code> , <code>acos_rel</code> , <code>atan_rel</code>	accurate, or one double beyond; a bound of x beyond 2^{52} is kept
<code>div_rel</code> , <code>%</code>	accurate; tightest where the bounds of the quotients are doubles
<code>cancel_minus</code> , <code>cancel_plus</code>	tightest
<i>Set operations, constructors, numeric and boolean functions</i>	
<code>&</code> , <code> </code> , <code>interval(l, u)</code>	tightest (exact)
<code>textToInterval(s)</code> , <code>operator>></code>	tightest for the literals of IEEE 1788-2015; valid for expressions, computed with the operations above
<code>midpoint()</code> , <code>width()</code> , <code>rad()</code> , <code>mag()</code> , <code>mig()</code>	as 12.12.8 requires
<code>mid()</code> , <code>hausdorff()</code>	tightest
<code>relations</code>	exact, as Tables 10.3 and 10.4
<code>operator<<</code>	hexa : exact, read back bit for bit. Decimal formats: valid, and the tightest with the digits asked for where the C library rounds to nearest as it should

Where a bound of J is one double from the image of a bound of the preimage, the reverse functions keep a point of I whose image is just outside J : `sqrt_rel` $([1 + 2^{-52}, +\infty], [-1, 1 + 2^{-52}])$ is $[-1, 1 + 2^{-52}]$, whose tightest enclosure is $[1, 1 + 2^{-52}]$. They are accurate all the same, the widened J of `nextOut` holding that image.

Before GAOL v5, the elementary functions were correctly rounded to nearest and moved one double outward, which is at most one double beyond the tightest bound, and the hyperbolic functions were taken from the mathematical library of the system, whose values are sometimes further than one double from the exact ones.

13

Floating-point numbers

13.1 Floating-point constants

In addition to the constants available through `numeric_limits<double>`, GAOL defines the following `double` constants:

Constant (double)	Value
<code>two_pi</code>	$\downarrow 2\pi \uparrow$
<code>pi</code>	$\downarrow \pi \uparrow$
<code>half_pi</code>	$\downarrow \frac{\pi}{2} \uparrow$
<code>pi_dn</code>	$\downarrow \pi \downarrow$
<code>pi_up</code>	$\uparrow \pi \uparrow$
<code>half_pi_dn</code>	$\downarrow \frac{\pi}{2} \downarrow$
<code>half_pi_up</code>	$\uparrow \frac{\pi}{2} \uparrow$
<code>ln2_dn</code>	$\downarrow \ln 2 \downarrow$
<code>ln2_up</code>	$\uparrow \ln 2 \uparrow$
<code>two_power_51</code>	2^{51}
<code>two_power_52</code>	2^{52}
<code>GAOL_NAN</code>	NaN (quiet)
<code>GAOL_INFINITY</code>	$+\infty$

13.2 Floating-point functions

`bool feven (double x)`

Returns true whenever `x` is even.

This function should not be used with infinity and NaN arguments.

Example	
1	<code>cout << feven(3.0) << ' ' << feven(4.0) << ' ' << feven(3.5);</code>
2	<code> false true false</code>

`double next_float (double x)`

Returns the smallest `double` greater than `x`.

double previous_float (double x)

Returns the greatest **double** smaller than x.

bool is_signed (double x)

Returns true whenever x is signed. No provision is made concerning the fact that x is a NaN. If you only want to test for negative numbers (and -0), you will have to test also whether x is a NaN by using `std::isnan()`.

double minimum (double x, double y)

Returns the minimum **double** value of x and y. This function is commutative and returns -0 when comparing -0 and +0, i.e.:

$$\begin{cases} \min(x, y) &= \min(y, x), & \forall x \neq \text{NaN}, \forall y \neq \text{NaN} \\ \min(x, \text{NaN}) &= \min(\text{NaN}, x) = \text{NaN}, & \forall x \\ \min(-0, 0) &= \min(0, -0) = -0 \end{cases}$$

double maximum (double x, double y)

Returns the maximum **double** value of x and y. This function is commutative and returns +0 when comparing -0 and +0, i.e.:

$$\begin{cases} \max(x, y) &= \max(y, x), & \forall x \neq \text{NaN}, \forall y \neq \text{NaN} \\ \max(x, \text{NaN}) &= \max(\text{NaN}, x) = \text{NaN}, & \forall x \\ \max(-0, 0) &= \max(0, -0) = 0 \end{cases}$$

ULONGLONGINT

macro

Macro standing for an unsigned integral data type with a size equal to 8 bytes (usually `unsigned long long int`).

ULONGLONGINT nb_fp_numbers (double a, double b)

Returns the number of floating-point numbers in the interval $[a, b]$. In particular, we have:

- `nb_fp_numbers(a, next_float(a)) == 2`
- `nb_fp_numbers(a, a) == 1`

Note. As a precondition, a shall be lower or equal to b.

Returns `numeric_limits<ULONGLONGINT>::max()` if either a or b is a NaN or an infinity, or if a is greater than b. In addition, raises an `invalid_action_error` exception (see Section 17.1, page 80) or calls `gaol_error` depending on the way the library was configured.

```
Example
1 cout << nb_fp_numbers(1.0, next_float(1.0)) << ' '
2   << nb_fp_numbers(1.0, 2.0);
3  2 4503599627370497
```

14

Manipulating the FPU

GAOL provides functions to set the rounding direction of the floating-point unit. The operations of GAOL do not need them: each sets the rounding direction upward itself (see Section 3.2, page 13).

On x86 processors, with GCC, Clang and Visual C++, they write the control registers of the x87 and SSE units directly (`fnstcw/fldcw`, `stmxcsr/lldmxcsr`), rather than calling `fesetround()`, which cost 50 to 250 ns per call with the C runtime of Windows; with Visual C++ for x64, the SSE register only, the x87 unit being unused there. [GAOL v5] Elsewhere (ARM, POWER, s390x, RISC-V), they call `fesetround()`.

14.1 Rounding functions

`void round_downward (void)`

Sets the rounding direction mode towards $-\infty$.

`void round_nearest (void)`

Sets the rounding direction mode to the nearest/even.

`void round_upward (void)`

Sets the rounding direction mode to $+\infty$.

14.2 Saving the rounding direction

`unsigned short get_fpu_cw (void)`

Returns the rounding direction, as `fegetround()` of `<fenv.h>` gives it.

`void reset_fpu_cw (unsigned short st)`

Sets the rounding direction *st*, which `get_fpu_cw()` returned, with `fesetround()`.

GAOL 4 read and wrote the control word of the x87 unit with these functions on x86 Linux and macOS, which left the direction of the SSE instructions alone, and 16 bits of the control register on 64-bit ARM, which left out its rounding bits. [\[GAOL v5\]](#)

15

Version information

The library provides four constants to allow programs to determine at runtime with which version they are dynamically linked with. The versioning scheme adopted is the one used by the Apache Software Foundation described at <http://apr.apache.org/versioning.html>.

`unsigned int version_major` `const`

Major version of the library: 5 for GAOL v5.

`unsigned int version_minor` `const`

Minor version of the library.

`unsigned int version_micro` `const`

Patch version of the library.

`const char *const version` `const`

Version of the library as a string.

Example

```
1 cout << gaol::version;  
2  5.0.0
```


16

Additional functions

The following functions are utility functions not necessarily related to intervals or floating-point numbers.

```
template <typename T>  
bool odd (const T& x)
```

Returns **true** if x is odd and false otherwise. The T type may be any type providing the **&** (“bitwise **and**”) operator with the same semantics as the one for **ints**.

```
template <typename T>  
bool even (const T& x)
```

Returns **true** if x is even and false otherwise. The T type may be any type providing the **&** (“bitwise **and**”) operator with the same semantics as the one for **ints**.

17

Error handling

A program that uses GAOL may report errors in two different ways:

- by throwing an exception;
- or by printing a message and aborting.

The mechanism in use depends on the way the library is configured. If you use the option `--enable-exceptions=yes`, the default, all errors are reported through exception throwing; otherwise, a message is printed to the standard error, and the program aborts. The CMake build always reports errors by exceptions. Relying on exceptions is more in the C++ spirit, though it may incur some overhead.

It is up to the user to comply with this mechanism when adding error reporting code to ones program. GAOL defines the following macro to be used whenever one wants to report an error.

gaol_ERROR (*excep,msg*) macro

The behavior of the macro depends on the value chosen for the option `--enable-exceptions`: if exceptions are enabled, exception `gaol_core::excep` is raised with the message *msg*; otherwise, the program aborts with message *msg*.

Example

```
1 interval x;
2
3 [Code manipulating x]
4
5 if (x.is_empty()) {
6     gaol_ERROR(invalid_action_error,"Emptiness of one interval");
7 }
```

The `gaol_error()` function is defined as follows:

void gaol_error (const char *const *err*)
void gaol_error (const char **file*, int *line*, const char **err*)

Displays a message on the standard error output. The ternary version should be called with the `GAOL_FILE_POS` macro for the first two parameters.

The `GAOL_FILE_POS` macro is described in the next section.

17.1 Exceptions

The library defines `gaol_exception` as a class to be used as a base class for all GAOL exceptions. All of them provide at least the name of the file and the line number from where the exception has been thrown. As a facility, GAOL defines the following macro:

GAOL_FILE_POS

macro

Expands itself into the first two arguments of any constructor for `gaol_exception` or one of its derived classes:

Example

```
1 if ([some condition]) {  
2     throw gaol_exception(GAOL_FILE_POS,  
3                           "No additional information");  
4 }
```

All GAOL exceptions can be sent to an output stream through the “<<” operator.

17.1.1 The `gaol_exception` exception

The `gaol_exception` class is the base class from which derive all GAOL exceptions. It inherits from the C++ standard class `exception`.

Every exception class deriving from it must at least provide the name of the file and the line where the corresponding exception was thrown. As a consequence, the constructors for `gaol_exception` are as follows:

```
gaol_exception::gaol_exception (const char* f, unsigned l)  
gaol_exception::gaol_exception (const char* f, unsigned l,  
                                const char* e)
```

Constructs a `gaol_exception` being thrown from file *f* at line *l*. The second form permits adding some explanatory string *e*.

The class offers the following accessors:

```
const char* gaol_exception::file (void) const
```

Return the name of the file from where the exception was thrown.

```
unsigned int gaol_exception::line (void) const
```

Returns the line number in the file from where the exception was thrown.

```
const char* const gaol_exception::explanation (void) const
```

Returns a string explaining why the exception was thrown. Returns an empty string if no additional information was provided.

17.1.2 The `input_format_error` exception

The `input_format_error` exception is thrown whenever one attempts to create an interval from an invalid string. This situation may occur when reading an interval from a stream with the `>>` operator, or when reading a string with `gaol::textToInterval()` or the constructor from two strings.

This class, as all GAOL exceptions, derives from `gaol_exception` (see Section 17.1.1, page 80). Its constructors have the same format than the ones for `gaol_exception`, namely:

```
input_format_error::input_format_error (const char* f, unsigned l)
input_format_error::input_format_error (const char* f, unsigned l, const char*
e)
```

Constructs an `input_format_error` being thrown from file *f* at line *l*. The second form permits adding some explanatory string *e*.

The methods of the class are inherited from `gaol_exception` (see Section 17.1.1, page 80).

Example

```
1 try {
2     interval x = textToInterval("sin(1)+");
3 } catch (const gaol::input_format_error& e) {
4     cout << e.explanation();
5 }
6  Syntax error in interval initialization: sin(1)+
```

17.1.3 The `unavailable_feature_error` exception

This exception is thrown whenever an unavailable feature is requested. No operation of GAOL v5 throws it: `atan2()`, which did, is implemented. [GAOL v5]

This class, as all GAOL exceptions, derives from `gaol_exception`. (see Section 17.1.1, page 80). Its constructors have the same format than the ones for `gaol_exception`, namely:

```
unavailable_feature_error::unavailable_feature_error
                                (const char* f, unsigned l)
unavailable_feature_error::unavailable_feature_error
                                (const char* f, unsigned l, const char* e)
```

Constructs an `unavailable_feature_error` being thrown from file *f* at line *l*. The second form permits adding some explanatory string *e*.

The methods of the class are inherited from `gaol_exception` (see Section 17.1.1, page 80).

17.1.4 The `invalid_action_error` exception

This exception is thrown whenever a function is called with invalid arguments (e.g. calling `nb_fp_numbers()` with NaNs as parameters).

This class, as all GAOL exceptions, derives from `gaol_exception` (see Section 17.1.1, page 80). Its constructors have the same format than the ones for `gaol_exception`, namely:

```
invalid_action_error::invalid_action_error (const char* f,
                                              unsigned l)
invalid_action_error::invalid_action_error (const char* f,
                                              unsigned l, const char* e)
```

Constructs an `invalid_action_error` being thrown from file *f* at line *l*.
The second form permits adding some explanatory string *e*.

The methods of the class are inherited from `gaol_exception` (see Section 17.1.1, page 80).

17.2 Warnings

```
void gaol_warning (const char *warn)  
void gaol_warning (const char *file, int line, const char *warn)
```

Prints the message *warn* on the standard error output. The second form should be called with the `GAOL_FILE_POS` macro for the first two parameters.

18

Debugging facilities

The debugging facilities described hereunder are available only if GAOL has been configured with the debugging facilities enabled, which the Debug build of each build does: `-DCMAKE_BUILD_TYPE=Debug` with CMake, the `--enable-debug` option of `configure` (see Section 2.5, page 7), or `--buildtype=debug` with meson. They define `GAOL_DEBUGGING` in `gaol/gaol_configuration.h`: GAOL then checks its assertions.

`int debug_level`

Global variable used to remember the current value of the debugging level. This variable is set when initializing the library. The variable is declared in the `gaol_core` namespace, which `gaol` imports.

`GAOL_DEBUG (lvl,cmd)`

macro

Executes `cmd` if `lvl` is lower or equal to the current debugging level (see the variable `debug_level` above).

This macro defaults to nothing if the library was not configured with the debugging facilities.

A possible use for this macro is as follows:

Example

```
1 interval x(-10,10);
2
3 [Some code]
4
5 GAOL_DEBUG(1,cout << "The value of x is " << x);
6 x += interval(3.5,4.5);
7 GAOL_DEBUG(2,cout << "Now the value of x is " << x);
```

The first message will be displayed whenever the debugging level is at least 1, and the second one only if it is at least 2.

`GAOL_ASSERT (cond)`

macro

Tests whether `cond` holds. Aborts with an error message if it is not the case.

This macro defaults to nothing if the library was not configured with the debugging facilities.

A possible use for this macro is as follows:

Example

```
1 int x;  
2 cout << "Give an integer no greater than 5: ";  
3 cin >> x;  
4 GAOL_ASSERT(x <= 5);
```

19

Profiling

The following functions permit computing the time used for a computation. The returned times are *user* times, meaning that delays induced by input/output operations and freezing during CPU switches in multi-programming environments are not taken into account.

If you need to keep track of several events, consider using an object of the `timepiece` class (see Section 19.1, page 86) instead of calling directly the functions below.

Warning. The precision of the timing functions depends on the platform used. What is more, despite the fact that the reported times are user times, they may vary from an execution to another, and can get larger on heavily loaded machines. `tests/performance.cpp` shows how GAOL v5 measures the time of its operations, in nanoseconds.

long `get_time` (void)

Returns the time in milliseconds since a certain unspecified moment.

This function should only be used to compute differences between two calls since the starting point may vary depending on the availability of `clock()` or `getrusage()` on the system.

Warning. if the function in use is the standard `clock()`, the time returned will wrap approximately every 72 minutes. Consequently, it is not safe to use `get_time()` in that case for processes requiring more than 72 minutes to execute.

void `reset_time` (void)

Resets the time counter. To be called just before executing some code to be profiled.

long `elapsed_time` (void)

Returns the time in milliseconds elapsed between now and the last call to `reset_time()`.

long `intermediate_elapsed_time` (void)

Returns the time in milliseconds elapsed between now and the last call to `reset_time()` or to `intermediate_elapsed_time()`.

Here is a typical example of use of the timing functions:

Example

```
1 int main(void)
2 {
3     reset_time();
4     for (unsigned int i=0;i<1000;++i) {
5         [Some time consuming operations]
6     }
7     cout << "Elapsed time: " << elapsed_time() << " ms." << endl;
8     gaol::cleanup();
9     return 0;
10 }
```

19.1 The `timepiece` class

A `timepiece` object allows to keep track of the time spent to perform a particular task. Since the counter used is local to the object, it is possible to monitor more than one such process.

19.1.1 Methods of the `timepiece` class

`void timepiece::start (void)`

Starts the `timepiece`.

`void timepiece::stop (void)`

Stops the `timepiece` and accumulates the time spent since the last call to `start()`.

`void timepiece::reset (void)`

Resets to zero the counter keeping track of the total time the `timepiece` was running.

`long timepiece::get_total_time (void) const`

Returns the total amount of time the `timepiece` was running (time between calls to `start()` and `stop()`). The `timepiece` shall have been stopped by calling the `stop()` method before calling this one.

`long timepiece::get_intermediate_time (void) const`

Returns the amount of time spent since the last call to `start()`.

Example

```
1 int main(void)
2 {
3     timepiece t;
4     t.start();
5     for (unsigned int i=0;i<1000;++i) {
6         [Some time consuming operations]
7         cout << "Intermediate time: "
8             << t.get_intermediate_time() << " ms." << endl;
9     }
10    t.stop();
11    cout << "Elapsed time: " << t.get_total_time() << " ms." << endl;
12    gaol::cleanup();
13    return 0;
14 }
```


20

Additional Documentation

20.1 Documentation on GAOL

The primary reference is this manual. The directory `doc` of the sources holds the pages that give the details of GAOL v5, with the measures behind its choices:

- `doc/building.md`: the CMake, autotools and meson builds, and their options;
- `doc/using.md`: the flags of interval arithmetic, GAOL from CMake and from pkg-config, the namespaces and the rounding direction;
- `doc/three-builds.md`: what the three builds agree on, what the options are worth, and the compilers and options they refuse;
- `doc/tests.md`: what the programs of `tests` check;
- `doc/accuracy.md`: the tightness of each operation, with its algorithm and what the tests found;
- `doc/differences.md`: each change GAOL v5 brings to GAOL 4.2.2, and where it comes from;
- `doc/continuous-integration.md`: the systems, processors and compilers GAOL is built and tested on;
- `doc/compare/README.md`: GAOL compared with libieeep1788, filib++, PROFIL/BIAS and Solaris Studio, their special cases against IEEE 1788-2015 and their performance;
- `3rd/README.md`: the version of CORE-MATH GAOL takes, and what differs from it.

The manual of GAOL 4, by Frédéric Goualard, is kept in `manual/v4`.

20.2 References

The following articles, books and standards have inspired in some way or another the devising of the GAOL library and/or the writing of this manual.

- *IEEE Standard for Interval Arithmetic*. IEEE Std 1788-2015 [9].
- *The CORE-MATH Project*. Alexei Sibidanov, Paul Zimmermann and Stéphane Gloudu [13], and <https://core-math.gitlabpages.inria.fr/>.
- *How do you compute the midpoint of an interval?* Frédéric Goualard [5].
- *Fast and Correct SIMD Algorithms for Interval Arithmetic*. Frédéric Goualard [4].
- *Interval Extensions of Multivalued Inverse Functions*. Frédéric Goualard [3].
- *Interval Arithmetic Specification*. Dmitri Chiriaev and G. William Walster. Draft revised May 1998.
- *The Extended Real Interval System*. G. William Walster. April 1998.
- *C++ Interval Arithmetic Programming Reference*. Sun Microsystems, Inc. October 2000, revision A.
- *Interval Arithmetic: From Principles to Implementation*. T. Hickey, Q. Ju, and M. H. van Emden [7].

21

Reporting bugs

Bugs and suggestions for improvement of GAOL v5 shall be submitted as issues of its repository:

<https://github.com/Jordan08/GAOL/issues>

A report is most useful with the program that shows the problem, the interval it gives and the one expected, the system, the processor, the compiler and the options GAOL was built with.

22

Contributors

The main implementor and lead designer for the GAOL library is Frédéric Goualard (<https://frederic.goualard.net>).

GAOL v5 is written by Jordan Ninin (ENSTA), for Codac.

The `interval pow(const interval&, const interval&)` function was designed by Marc Christie.

The code for the multiplication and the division is largely inspired from the one presented by Tim Hickey, Qun Ju and Maarten Van Emden in *Interval Arithmetic: from Principles to Implementation*, Journal of the ACM 48(5):1038–1068, september 2001.

GAOL v5 takes up:

- the patch IBEX applies to GAOL, by Gilles Chabert (initialization of the floating-point unit, rounding direction of `midpoint()` and `operator<<`);
- the fork of GAOL by Fabrice Le Bars, for Visual C++, MinGW and the systems the `configure` of GAOL does not know;
- the CMake build of GAOL in IBEX, by Cyril Bouvier and Gilles Chabert, with the compilation flags of the IBEX fork of Fabrice Le Bars;
- the rounding tests of Codac, which its tests follow.

Its elementary functions are those of CORE-MATH, by Alexei Sibidanov, Paul Zimmermann and the other authors of the project (Inria).

23

Moving a program from GAOL 4 to GAOL v5

Most programs written for GAOL 4 compile and run with GAOL v5 once the namespace is opened and `textToInterval()` reads the strings they gave to `interval(const char*)`. This chapter lists what a program may have to change, and what it will see differently. The page `doc/differences.md` gives each change with its reasons and its measures.

23.1 Compiling and linking

- `gaol/gaol.h` no longer opens the namespace `gaol`: add `using namespace gaol;`, or name `gaol::interval` (see Section 3.3, page 14). The type and the functions are in `gaol_core`, and the symbols of the library name it: a program compiled against the headers of GAOL 4 is compiled again.
- GAOL is the only library to link: the mathematical library GAOL 4 was built with is no longer used, CORE-MATH being compiled into GAOL, and the options of the builds that chose it are gone.
- The code including the headers of GAOL is compiled with the flags of interval arithmetic, which `gaol::gaol` and `gaol.pc` give (see Section 3.1, page 12); `gaol/gaol_config.h` refuses the options and the compilers that give wrong bounds (see Section 2.3.2, page 4).
- The intervals of floats, `gaol::intervalf` and `gaol::interval2f`, are compiled by none of the builds, which have no option for them: only a developer of GAOL compiles them, defining `GAOL_FLOAT_INTERVALS` in `gaol/gaol_config.h`.
- The functions of doubles rounded downward and upward, `exp_dn()`, `sin_up()` and the others of `gaol/gaol_double_op.h`, are no longer part of the interface: the header is no longer installed.
- The static intervals `interval::cst_one`, `cst_pi` and the other `cst_` members are gone: `interval::one()`, `interval::pi()` and the other functions give these constants, right even in the initialization of a static object (see Section 4, page 17).

- `gaol::init()` need not be called again after the program changed the rounding direction: each operation sets it upward itself (see Section 3.2, page 13). It need not be called at all, GAOL initializing itself with every compiler, Visual C++ included (see Section 4, page 17).
- The program calls `gaol::cleanup()` right after its last use of GAOL: unless the rounding direction is preserved, it sets back the rounding direction that the initialization found, which GAOL 4 left upward (see Section 4, page 17). It no longer frees what the initialization allocated, which the automatic cleanup frees when the program ends.
- The operators `==` and `!=` on intervals are gone, with the possibly relations (`possibly_eq()`, `possibly_le()`...), `certainly_eq()` and `certainly_neq()`, and so is the option `--enable-relations` of `configure` and `meson`: a program compares intervals with `set_eq()`, `set_disjoint()` and `!set_disjoint()`, and `<`, `<=`, `>` and `>=` are the certainly relations (see Section 7, page 27).
- The constructor from one string, `interval(const char*)`, is gone, and with it the conversion of a string wherever an interval is expected: `textToInterval("0.1")` reads a string in its place, and throws `input_format_error` as it did; `x = "[1, 2]"` becomes `x = textToInterval("[1, 2]")`, and `x + "0.1"` becomes `x + textToInterval("0.1")` (see Section 5.1, page 19). The constructor from two strings stays. `interval(0)`, which the constructor from one string made ambiguous, compiles.

23.2 Results that change

- **Static objects.** The intervals computed in the initialization of the static objects of a program linked with the static library GAOL are right: `interval::pi()` was $[-0, 0]$ there, and $[1, 2]/[-1, 1]$ was $[-0, 0]$ rather than $[-\infty, +\infty]$, the constants of GAOL being computed after these objects, except with MinGW-w64 (see Section 4, page 17).
- **The bounds are tighter:** the elementary functions give the tightest bounds, rather than one double beyond, and are exact where their value is a double; the integer powers and the n th roots are computed from exact products; `sin`, `cos` and `tan` are within one double of the tightest bounds at every magnitude (Chapter 12).
- **The empty set.** The constructors give it for $[+\infty, +\infty]$, $[-\infty, -\infty]$, bounds in the wrong order and NaN bounds, and assigning an infinity to an interval empties it (see Section 5.1, page 19); the intersection of disjoint intervals is it, with NaN bounds; so are `log` of an interval holding no positive number, `invabs_rel` of a negative interval and `div_rel` with no solution in I . `width()` of the empty set is NaN rather than `-1`.
- **Powers.** `pow(x, p)` for a double p no longer truncates p ; `pow(x, y)` keeps the negative part of x for an integer exponent only, and takes the `pow` of IEEE 1788-2015 otherwise (see Section 8.1.2, page 39). `nth_root` of an odd order is defined on the whole real line, and takes a negative order.
- **The comparisons** follow IEEE 1788-2015 for the empty set and the infinite bounds: `certainly_leq()` and its companions are true when

either interval is empty, and `set_strictly_contains()` takes an infinite bound as beyond itself (see Section 7, page 27).

- **Input and output.** Numbers are read exactly, and the literals of IEEE 1788-2015 are read, in any case of letters; "[inf]" is the empty set. Every string is an expression, where an interval may stand wherever a number may ("1+[1,2]"), an uncertain number being no bound ("[5?1]" is refused), and an error anywhere stops the reading. The reader knows every function of GAOL, and `gaol_ieee1788::textToInterval` reads the names of IEEE 1788-2015 (`pown`, `rootn`...). The format `hexa` writes interval literals, which are read back bit for bit; the decimal formats round the bounds outward whatever the C library, and the format `agreeing` keeps the digits of each bound (Chapter 10).
- **New operations:** `atan2()` is implemented, and the functions of IEEE 1788-2015 GAOL did not provide are added (`exp2`, `hypot`, `sign`, `trunc`..., Chapter 8).

Library Copying

GNU LIBRARY GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright © 1991 Free Software Foundation, Inc. 675 Mass Ave, Cambridge, MA 02139, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the library GPL. It is numbered 2 because it goes with version 2 of the ordinary GPL.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users.

This license, the Library General Public License, applies to some specially designated Free Software Foundation software, and to any other libraries whose authors decide to use it. You can use it for your libraries, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library, or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link a program with the library, you must provide complete object files to the recipients so that they can relink them with the library, after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

Our method of protecting your rights has two steps: (1) copyright the library, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the library.

Also, for each distributor's protection, we want to make certain that everyone understands that there is no warranty for this free library. If the library is modified by someone else and passed on, we want its recipients to know that

what they have is not the original version, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that companies distributing free software will individually obtain patent licenses, thus in effect transforming the program into proprietary software. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License, which was designed for utility programs. This license, the GNU Library General Public License, applies to certain designated libraries. This license is quite different from the ordinary one; be sure to read it in full, and don't assume that anything in it is the same as in the ordinary license.

The reason we have a separate public license for some libraries is that they blur the distinction we usually make between modifying or adding to a program and simply using it. Linking a program with a library, without changing the library, is in some sense simply using the library, and is analogous to running a utility program or application program. However, in a textual and legal sense, the linked executable is a combined work, a derivative of the original library, and the ordinary General Public License treats it as such.

Because of this blurred distinction, using the ordinary General Public License for libraries did not effectively promote software sharing, because most developers did not use the libraries. We concluded that weaker conditions might promote sharing better.

However, unrestricted linking of non-free programs would deprive the users of those programs of all benefit from the free status of the libraries themselves. This Library General Public License is intended to permit developers of non-free programs to use free libraries, while preserving your freedom as a user of such programs to change the free libraries that are incorporated in them. (We have not seen how to achieve this as regards changes in header files, but we have achieved it as regards changes in the actual functions of the Library.) The hope is that this will lead to faster development of free libraries.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, while the latter only works together with the library.

Note that it is possible for a library to be covered by the ordinary General Public License rather than by this special one.

Terms and Conditions for Copying, Distribution and Modification

0. This License Agreement applies to any software library which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Library General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either

verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term “modification”.)

“Source code” for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library’s complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a. The modified work must itself be a software library.
- b. You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- c. You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- d. If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you

distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a “work that uses the Library”. Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a “work that uses the Library” with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a “work that uses the library”. The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a “work that uses the Library” uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the

Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also compile or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

- a. Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)
- b. Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.
- c. If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.
- d. Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:
 - a. Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.
 - b. Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.
8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.
9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.
10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.
11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.
13. The Free Software Foundation may publish revised and/or new versions of the Library General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE

QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public License).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the “copyright” line and a pointer to where the full notice is found.

one line to give the library's name and a brief idea of what it does.
Copyright (C) year name of author

This library is free software; you can redistribute it and/or modify it under the terms of the GNU Library General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Library General Public License for more details.

You should have received a copy of the GNU Library General Public License along with this library; if not, write to the Free Software Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a “copyright disclaimer” for the library, if necessary. Here is a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the library 'Frob' (a library for tweaking knobs) written by James Random Hacker.

signature of Ty Coon, 1 April 1990
Ty Coon, President of Vice

That's all there is to it!

The license of CORE-MATH

The sources of CORE-MATH, in 3rd/math-core, which GAOL compiles into its library, are distributed under the following license (MIT license):

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Bibliography

- [1] Götz Alefeld and Jürgen Herzberger. *Introduction to Interval Computations*. Academic Press Inc., New York, USA, 1983. Traduit par Jon Rokne de l'original Allemand '*Einführung In Die Intervallrechnung*'.
- [2] David Marc Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, March 1991.
- [3] Frédéric Goualard. Interval extensions of multivalued inverse functions. Technical Report hal-00288457, HAL, 2008.
- [4] Frédéric Goualard. Fast and correct SIMD algorithms for interval arithmetic. In *PARA '08, Lecture Notes in Computer Science 6126–6127*. Springer, 2012.
- [5] Frédéric Goualard. How do you compute the midpoint of an interval? *ACM Transactions on Mathematical Software*, 40(2), 2014.
- [6] Eldon Robert Hansen. *Global Optimization Using Interval Analysis*. Pure and Applied Mathematics. Marcel Dekker Inc., 1992.
- [7] Timothy J. Hickey, Qun Ju, and Maarten H. van Emden. Interval arithmetic: from principles to implementation. *Journal of the ACM*, 48(5):1038–1068, 2001.
- [8] IEEE. IEEE standard for binary floating-point arithmetic. Technical Report IEEE Std 754-1985, Institute of Electrical and Electronics Engineers, 1985. Reaffirmed 1990.
- [9] IEEE. *IEEE Standard for Interval Arithmetic*. Institute of Electrical and Electronics Engineers, 2015. IEEE Std 1788-2015, doi:10.1109/IEEESTD.2015.7140721.
- [10] Ramon Edgar Moore. *Interval Analysis*. Prentice-Hall, Englewood Cliffs, N. J., 1966.
- [11] Arnold Neumaier. *Interval methods for systems of equations*, volume 37 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 1990.
- [12] Helmut Ratschek and Jon Rokne. Interval methods. In *Handbook of Global Optimization*, pages 751–828. Kluwer Academic, 1995.
- [13] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondou. The CORE-MATH project. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*, pages 26–34, 2022.

- [14] Volker Stahl. *Interval Methods for Bounding the Range of Polynomials and Solving Systems of Nonlinear Equations*. Phd. thesis, Johannes Kepler Universität, Linz, September 1995.

Index

- $\downarrow\downarrow$, *see* downward rounding
- $\uparrow\uparrow$, *see* upward rounding
- `abs`, **51**
- accuracy
 - of the operations, **67**
- `acos`, **43**
- `acos_rel`, **46**
- `acosh`, **44**
- `acosh_rel`, **46**
- `acospi`, **43**
- `asin`, **43**
- `asin_rel`, **46**
- `asinh`, **44**
- `asinh_rel`, **46**
- `asimpi`, **43**
- assembler, **7**
- `atan`, **43**
- `atan2`, **43**
- `atan2pi`, **43**
- `atan_rel`, **46**
- `atanh`, **44**
- `atanh_rel`, **46**
- `atanpi`, **43**
-
- bison, **4**
-
- c++
 - standard, **4**
- `cancel_minus`, **48**
- `cancel_plus`, **48**
- cancellative operations, **48**
- canonical interval, **16**, **16**
- `ceil`, **52**
- cell, **vii**
- `certainly_ge`, **30**
- `certainly_geq`, **30**
- `certainly_le`, **30**
- `certainly_leq`, **30**
- `certainly_negative`, **31**
- `certainly_positive`, **30**
- `certainly_strictly_negative`, **31**
-
- `certainly_strictly_positive`, **31**
-
- `chi`, **51**
- `cleanup`, **18**
- CMake, **5**
 - using gaol from, **13**
- compiler
 - refused, **4**
- compiling
 - a file using gaol, **12**
- `configure`, **7**
- containement property, **13**
- CORE-MATH, **vii**, **41**
- `cos`, **42**
- `cosh`, **44**
- `cospi`, **43**
-
- `debug_level`, **83**
- debugging
 - option, **7**
- `div_rel`, **47**
- domain of a function, **37**
- downward rounding ($\downarrow\downarrow$), **13**
-
- `elapsed_time`, **85**
- empty
 - interval: constructing, **19**
 - interval: printing, **58**
 - interval: representation, **25**
- `[empty]`, **58**
- error
 - raising an exception, **7**
- `even`, **77**
- `exact_string`, **58**
- `exp`, **41**
- `exp10`, **41**
- `exp10m1`, **41**
- `exp2`, **41**
- `exp2m1`, **41**
- `explanation`, **80**
- `expm1`, **41**
-
- $\mathbb{F}$, **13**

- feven, **71**
- file, **80**
- flags of interval arithmetic, **12**
- flex, **4**
- floor, **52**
- fma, **39**
- format, **59**
- function
 - undocumented, **2**
- fused multiply-add, **6**
- gaol
 - namespace, **11**
 - pronunciation, **1**
 - web page, **3**
- GAOL 4
 - moving from, **95**
- GAOL v5, **1**
- GAOL_ASSERT, **83**
- GAOL_DEBUG, **83**
- gaol_ERROR, **79**
- gaol_error, **79**
- gaol_exception, **80**
- GAOL_FILE_POS, **80**
- gaol_warning, **82**
- get_fpu_cw, **73**
- get_intermediate_time, **86**
- get_time, **85**
- get_total_time, **86**
- hausdorff, **50**
- header
 - and namespace, *see* namespace
- help
 - on configuration, **7**
- hypot, **45**
- IEEE 1788-2015, **1**
 - names, **63**
 - relations, **27**
- init, **17**
- input_format_error, **81**
- integer, **53**
- intermediate_elapsed_time, **85**
- interval
 - canonical, *see* canonical interval
 - computation site (web), **2**
 - conversion to string, **58**
 - with infinite bounds, **21**
 - zero, **16**
- interval_format, **60**
- invabs_rel, **47**
- invalid_action_error, **81**
- inverse, **38**
- is_a_double, **33**
- is_an_int, **33**
- is_canonical, **34**
- is_common_interval, **34**
- is_empty, **34**
- is_entire, **34**
- is_finite, **35**
- is_signed, **72**
- is_symmetric, **35**
- is_zero, **34**
- left, **51**
- less, **31**
- license
 - of CORE-MATH, **vii**
- line, **80**
- log, **41**
- log10, **41**
- log10p1, **42**
- log1p, **42**
- log2, **41**
- log2p1, **42**
- mag, **50**
- max, **52**
- maximum, **72**
- meson, **8**
- messages
 - avoiding printing of, **7**
- mid, **51**
- mid_rad, **49**
- midpoint, **50**
- mig, **49**
- min, **52**
- minimum, **72**
- namespace
 - gaol, **11, 14**
 - gaol_core, **14**
 - gaol_ieee1788, **14, 63**
- nb_fp_numbers, **72**
- next_float, **71**
- nth_root, **40**
- nth_root_rel, **47**
- number
 - reading, **56**
- odd, **77**
- operator
 - relational, **1**
- operator*, **38**
- operator*=: **22**
- operator+, **38**
- operator+=, **22**

- operator-, 38
- operator==, 22
- operator/, 38
- operator/=: 23
- operator<, 32
- operator<=: 32
- operator<<, 58
- operator>, 32
- operator>=: 32
- operator>>, 55
- operator%, 38, 45
- operator% , 45
- operator%=, 23
- operator%= , 23
- operator&, 54
- operator&=: 22
- optimization
 - configuration option, 7
 - interprocedural, 6
- outward rounding, 13
- package, 6
- pkg-config, 13
- platforms, 3
- pow, 39
- power, 39
- precision, 60, 61
- previous_float, 72
- $\mathbb{R}$, 13
- rad, 49
- references
 - on interval arithmetic, 2
- relation
 - possibly (removed), 27
 - symbols, 32
- relational operator, 1
- reset, 86
- reset_fpu_cw, 73
- reset_time, 85
- reverse functions, *see* relational arithmetic
- right, 51
- round_downward, 73
- round_nearest, 73
- round_ties_to_away, 53
- round_ties_to_even, 53
- round_upward, 73
- Rounding
 - to nearest, 15
- rounding
 - direction, 13
 - downward, 13
 - outward, *see* outward rounding
- preserving, 6, 7, 14
 - upward, 13
- rsqrt, 45
- self, 22
- set_contains, 27
- set_disjoint, 28
- set_eq , 28
- set_ge, 29
- set_geq, 29
- set_le, 29
- set_leq, 29
- set_neq, 29
- set_strictly_contains, 28
- sign, 53
- sin, 42
- sinh, 44
- sinpi, 43
- smig, 50
- split, 53
- split_left, 53
- split_right, 54
- sqr, 39
- sqrt, 39
- sqrt_rel, 46
- SSE2, 5
- start, 86
- stop, 86
- straddles_zero, 33
- strictly_less, 32
- strictly_straddles_zero, 33
- string
 - converting interval to, *see* interval
- tan, 42
- tanh, 44
- tanpi, 43
- tests, 8
- textToInterval, 20
- this, 22
- tightness, 67
- tool
 - mandatory ~ to compile gaol, 4
- trunc, 53
- trust rounding mode, 14
- ULONGLONGINT, 72
- unavailable_feature_error, 81
- undocumented
 - use of ~ functions, 2
- upward rounding ($\uparrow\uparrow$), 13
- verbose mode, 6, 7

version, 75
version_major, 75
version_micro, 75
version_minor, 75

width, 49